

**Министерство науки и высшего образования Российской Федерации**  
Федеральное государственное автономное образовательное учреждение  
высшего образования

**«Пермский национальный исследовательский политехнический  
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и  
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

**ОТЧЕТ**

**Лабораторная работа №11**

**«Информационные динамические структуры»**

**По дисциплине «Основы алгоритмизации и программирования»**

Вариант 24

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2026

## ПОСТАНОВКА ЗАДАЧИ

Написать программу, в которой создаются динамические структуры и выполнить их обработку в соответствии со своим вариантом.

**Для каждого вариант разработать следующие функции:**

1. Создание списка.
2. Добавление элемента в список (в соответствии со своим вариантом).
3. Удаление элемента из списка (в соответствии со своим вариантом).
4. Печать списка.
5. Запись списка в файл.
6. Уничтожение списка.
7. Восстановление списка из файла.

## АНАЛИЗ

Рассмотрим реализацию структуры двунаправленного списка. Оставшиеся структуры получим путем её изменения.

1. Метод `pushBack` должен принимать на вход данные. Создается новый узел, в него записываются исходные данные. Для нового узла в качестве указателя на предыдущий элемент ставится ссылка на текущий хвост, а для хвоста указываем новый узел в качестве следующего. Затем перезаписываем хвост, присваивая ему значение нового узла. Также следует рассмотреть случай, при котором размер списка равен нулю, так как в таком случае мы не сможем обращаться к полям хвоста. В этом случае перезапишем голову и хвост списка новым узлом. В конце обоих случаев увеличим размер списка на единицу.

2. Метод `pushFront` аналогичен методу `pushBack`, за исключением того, что разворачивается порядок.

3. Для метода `popBack` запоминаем указатель на последний удаляемый элемент. Затем перезаписываем хвост своим предшествующим элементом. Далее проверяем: если хвост `!= nullptr`, то указываем в качестве следующего за ним элемента `nullptr`. В ином случае лист оказывается пустой, перезаписываем голову. Далее в любом случае освобождаем память и уменьшаем размер. Также нужно обработать граничный случай, когда размер списка уже ноль — в таком случае выходим из метода ничего не делая.

4. Метод `popFront` аналогичен `popBack`

5. Метод `insert` принимает на вход значение, которое мы хотим добавить и индекс этого значения в массиве. Создаем новый узел с входными данными. Если индекс равен нулю — вызываем `pushFront`, если равен размеру массива — вызываем `pushBack`. Далее следует проверить, лежит ли индекс на интервале  $(0; \text{size})$ , если нет — индекс невалиден. Далее задаем `node = head`, и в цикле перезаписываем `node = node->next` `idx` раз — так мы найдем узел, перед которым мы должны вставить данные. Далее для предшествующего `node` узла в поле `next` записываем указатель нашего нового узла, затем для нового узла в качестве `prev` указываем это узел. Затем для `node` указываем в качестве предыдущего значения новый узел, а для нового узла — в качестве следующего указываем `node`. Увеличиваем размер на единицу.

6. Метод `remove` принимает в качестве аргумента индекс удаляемого элемента. Проверяем, лежит ли он на полуинтервале  $[0, \text{size})$ . Если `idx == 0`, то вызываем `popFront`, если `idx == size - 1` — вызываем `popBack`. В иных случаях аналогично методу `insert` ищем узел с нужным индексом. Задаем `node->next->prev = node->prev` и `node->prev->next = node->next`, удаляем искомый узел и уменьшаем размер.

7. Метод `get` ищет элемент с нужным индексом тем же способом, что используются в `remove` и `insert` и возвращает его.

8. Метод `find` идет по узлам с головы до тех пор, пока не встретит `nullptr`. На каждом шаге проверяется, равно ли значение в узле искомому и

счетчик  $i$  увеличивается на единицу. Таким образом  $i$  — индекс искомого элемента.

9. Метод `print` аналогично методу `remove` идет по всем элементам списка и выводит их значения в поток.

10. Функция `clear` аналогично методам `remove` и `print` перебирает все элементы и удаляет их. Затем значениям `head`, `tail` присваивается `nullptr`, а размер становится равен единице.

Для однонаправленного списка удаляется поле `prev` из узла, и соответственно вся работа с ним тоже. Отдельные изменения:

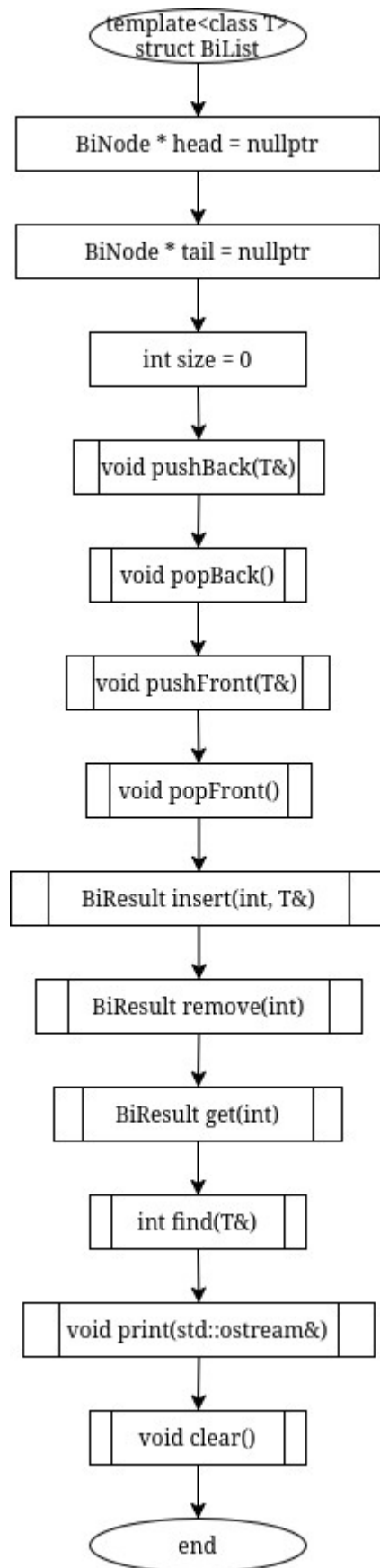
1. В методе `popBack` при размере равном одному, удаляем область памяти, лежащей по `head`, приравниваем концы списка к `nullptr` и обнуляем размер. В ином случае, начиная с `head`, перебираем узлы до тех пор, пока следующим узлом нашего списка не будет `tail`. Затем для найденного узла указываем `next` как `nullptr` и удаляем `tail`.

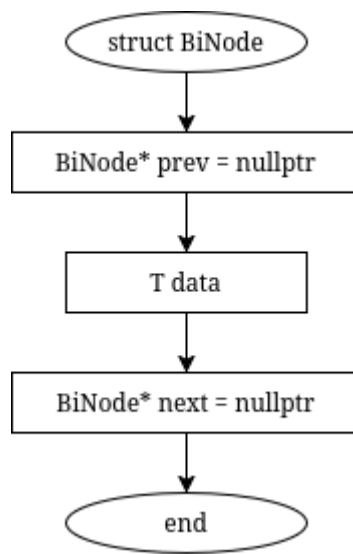
2. В методе `remove` так же не доходим на единицу до искомого узла, чтобы не потерять доступ (у нас нету `prev`, все пропущенные узлы теряются).

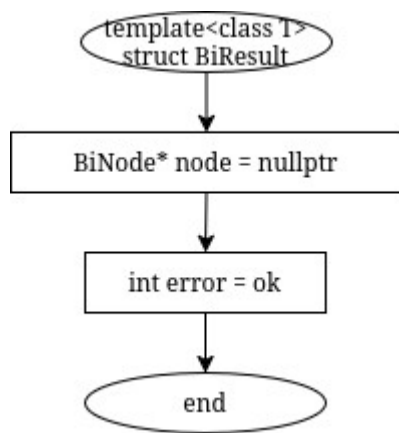
Очередь реализуем на основе односвязного списка. Оставляем методы `pushBack`, `pushFront`, `insert`, `print`, `clear` и добавляем `peek` — получение текущего `head`.

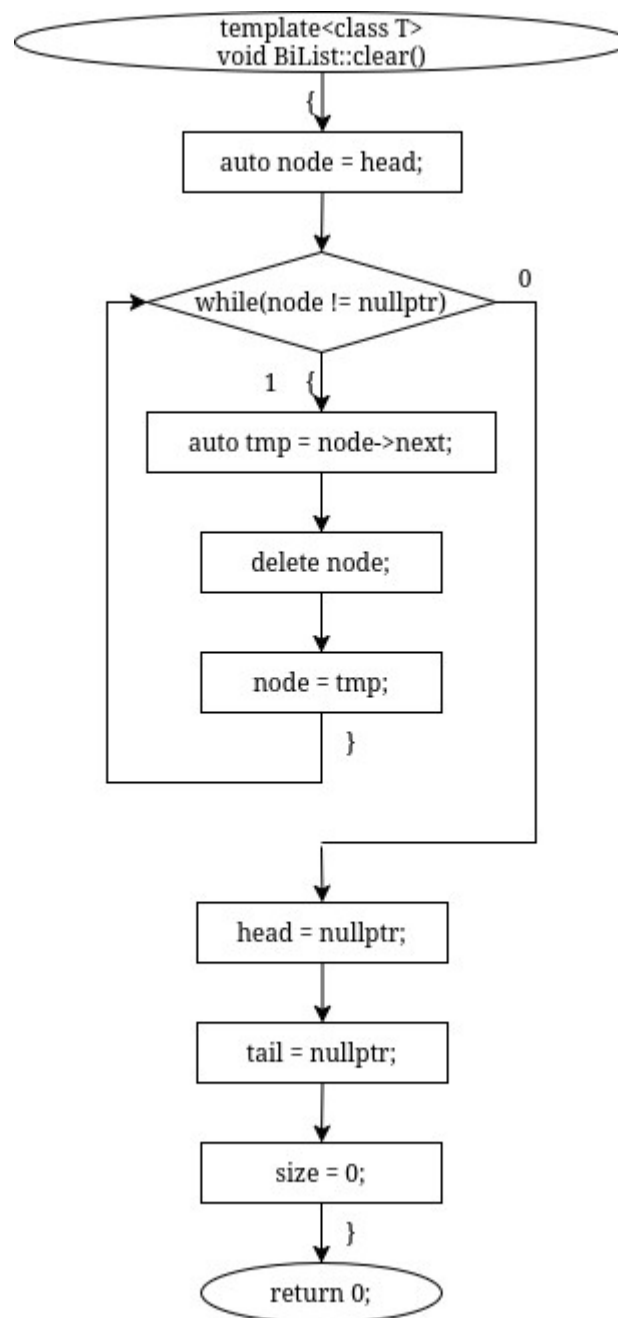
Стек реализуем также на основе односвязного списка. Оставляем функции `pushFront` (для простоты назовем её `push`), `popFront` (`pop`), `peek`, `print`, `clear`.

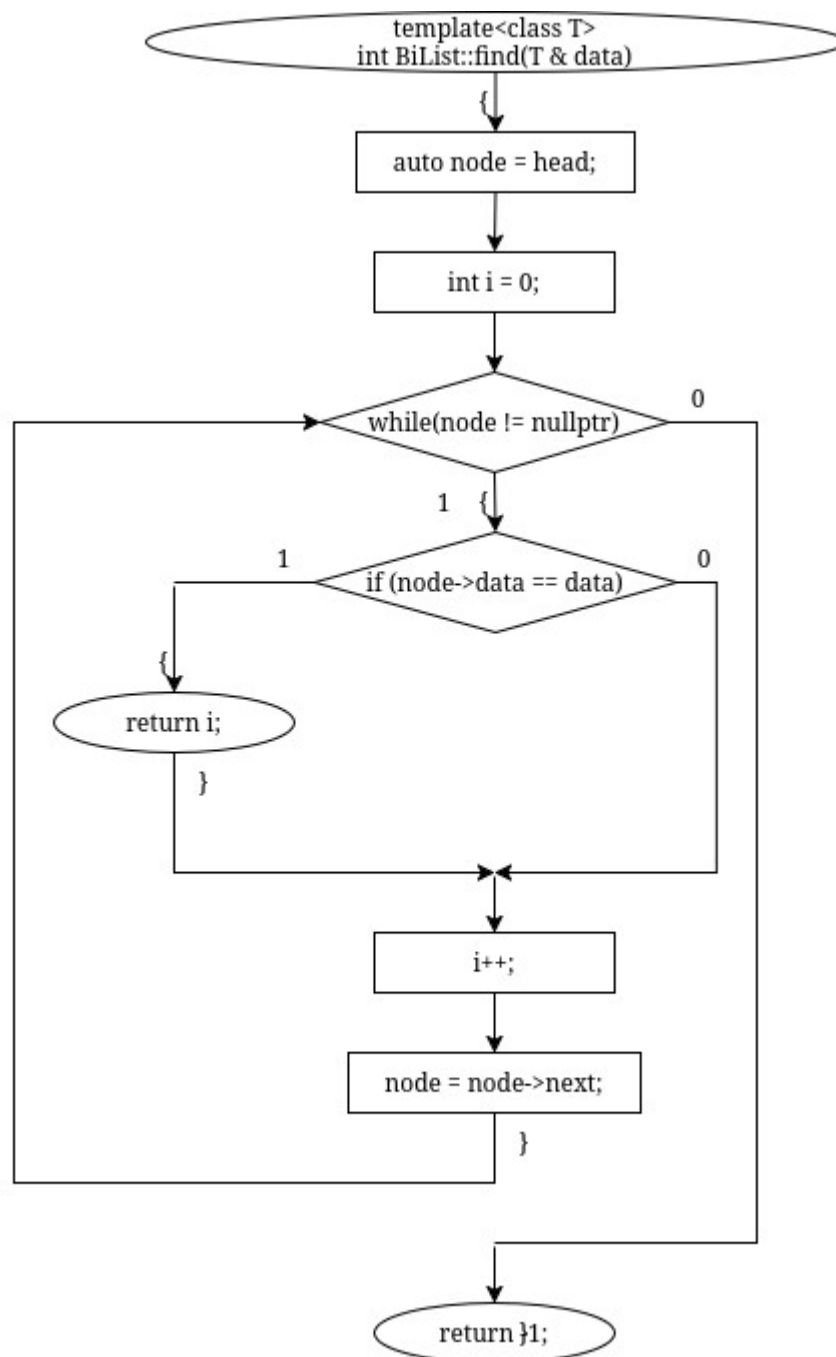
## БЛОК-СХЕМЫ

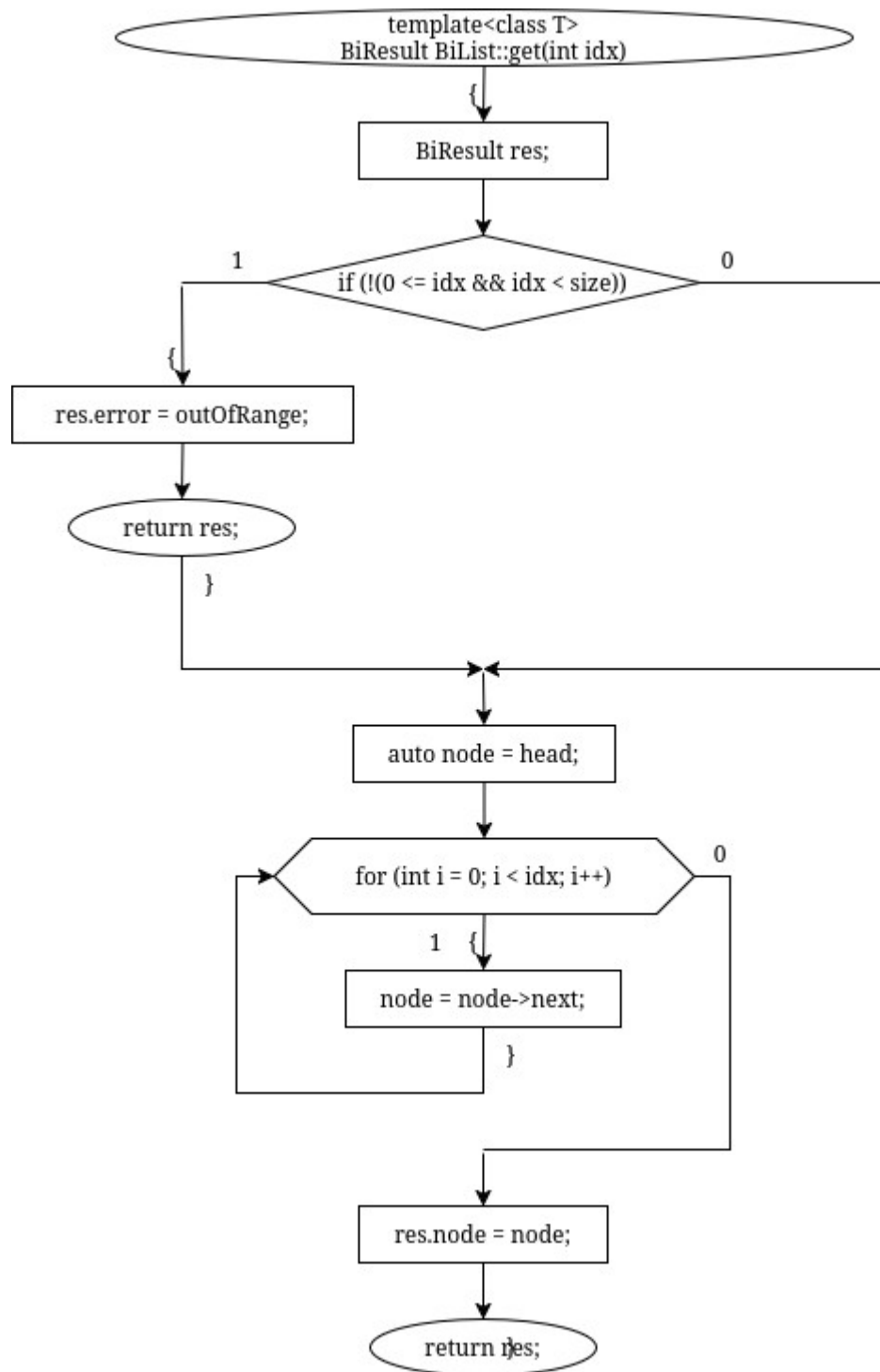


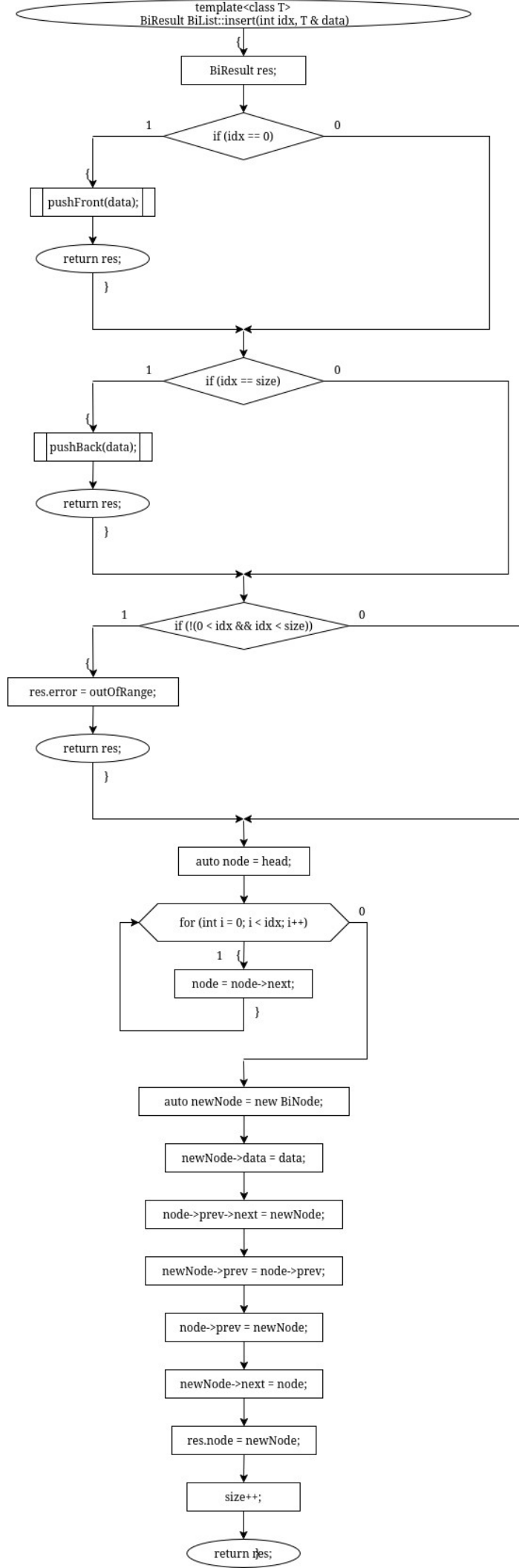


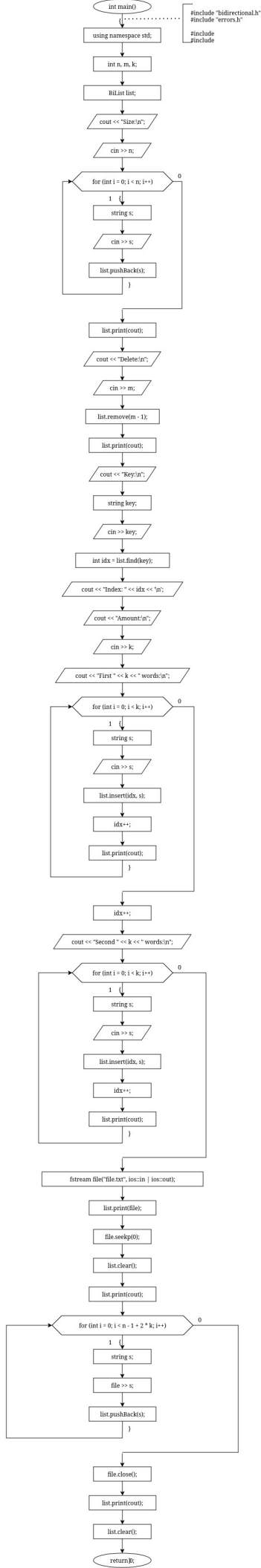


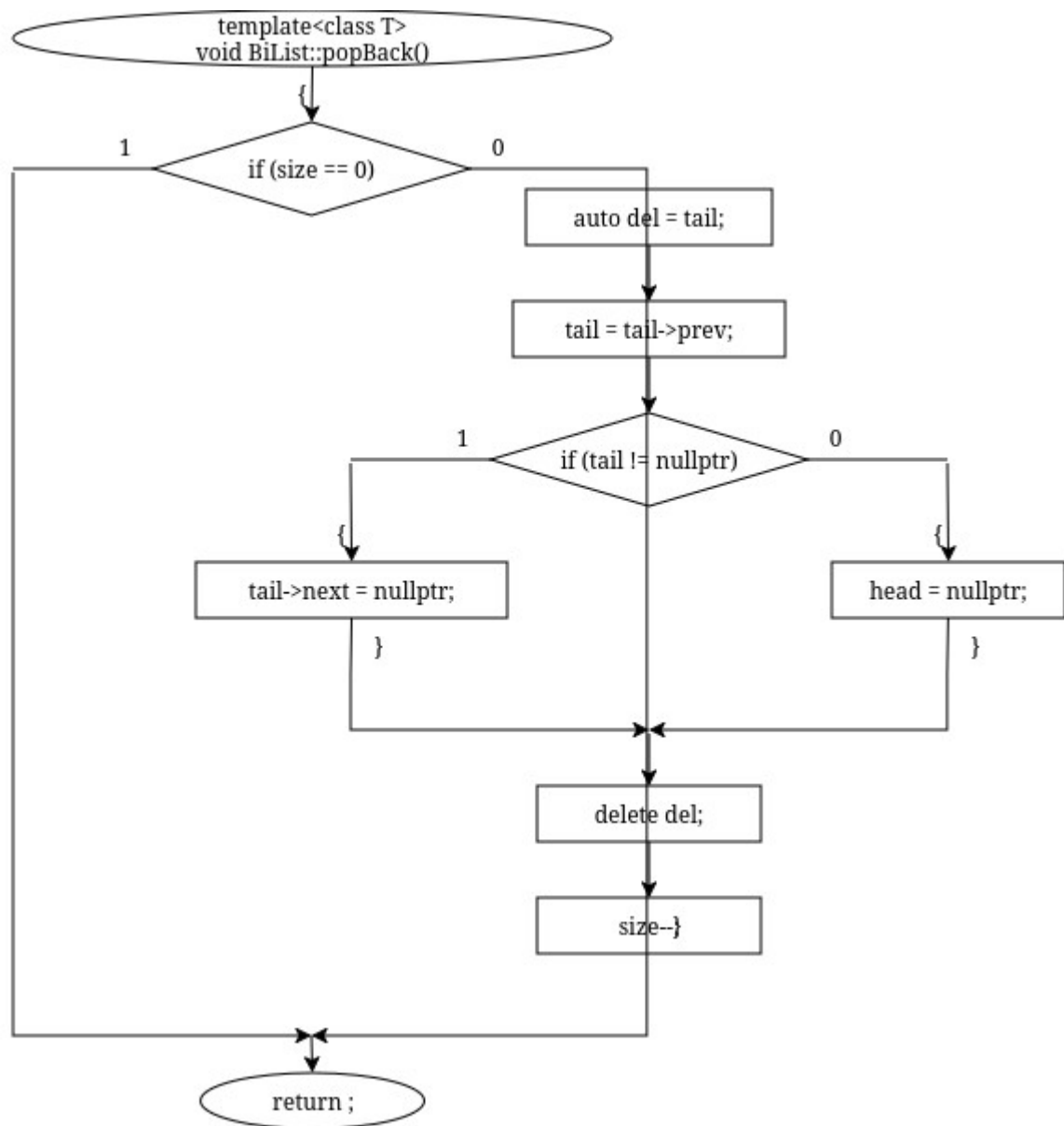


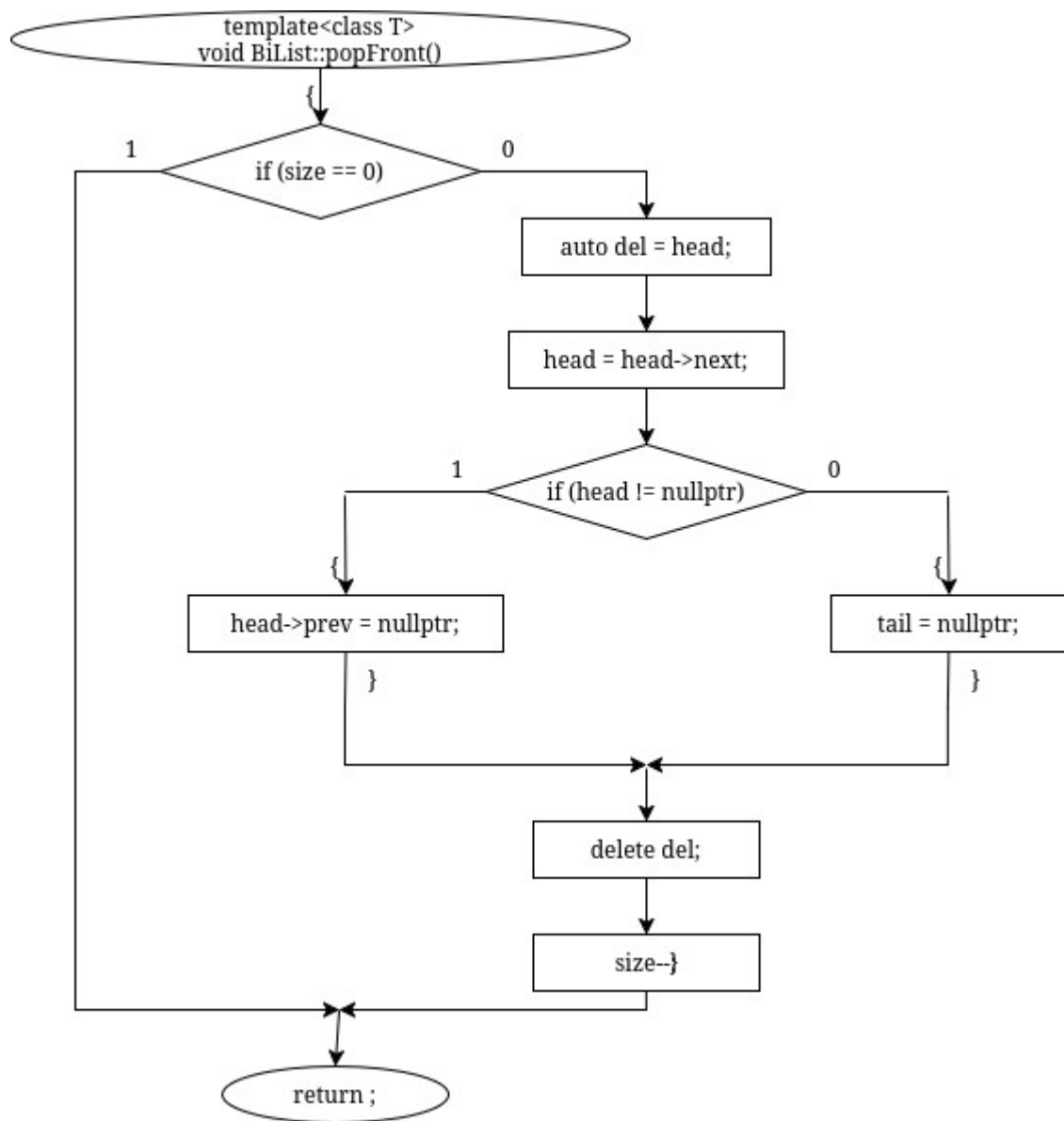


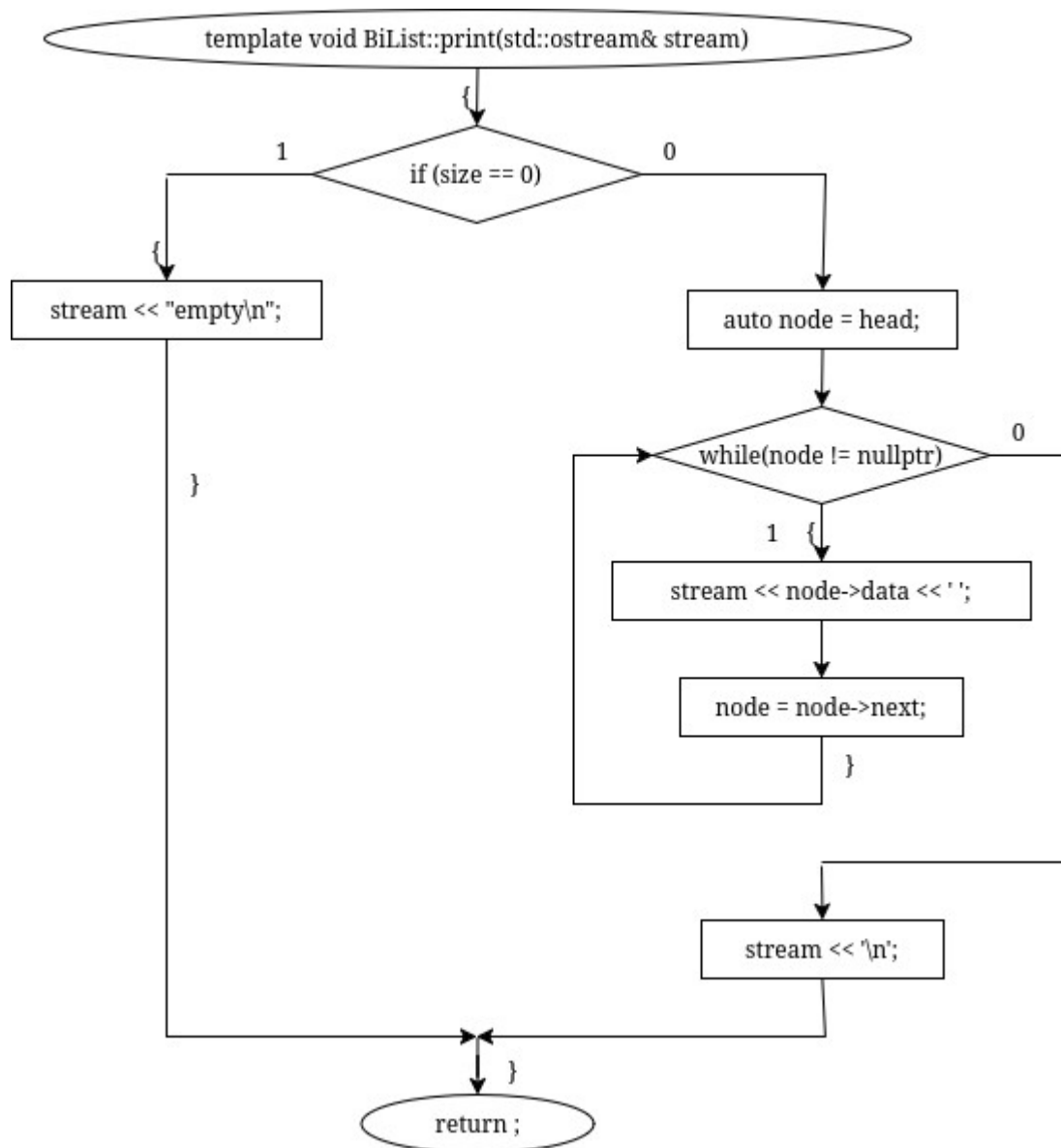


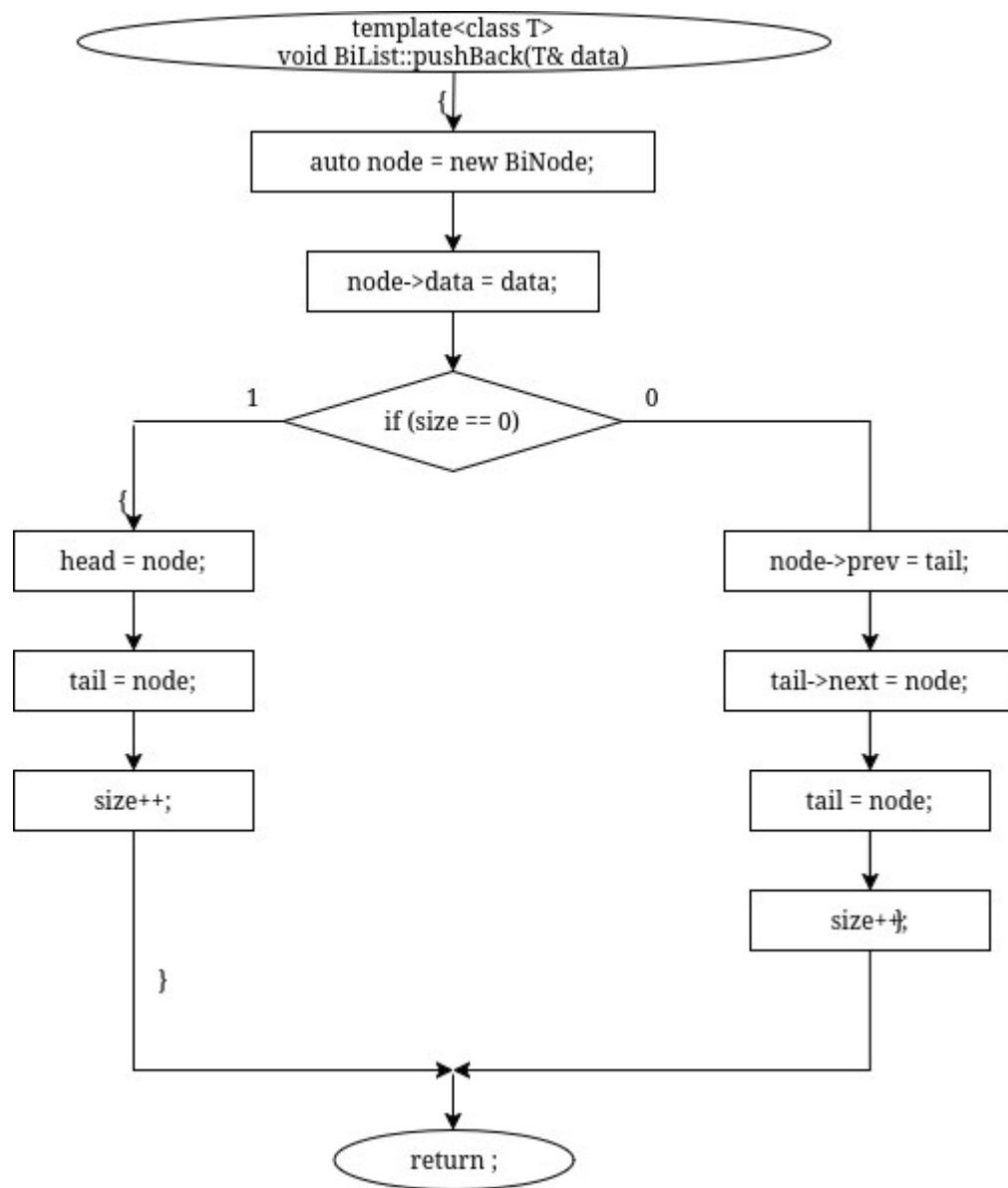


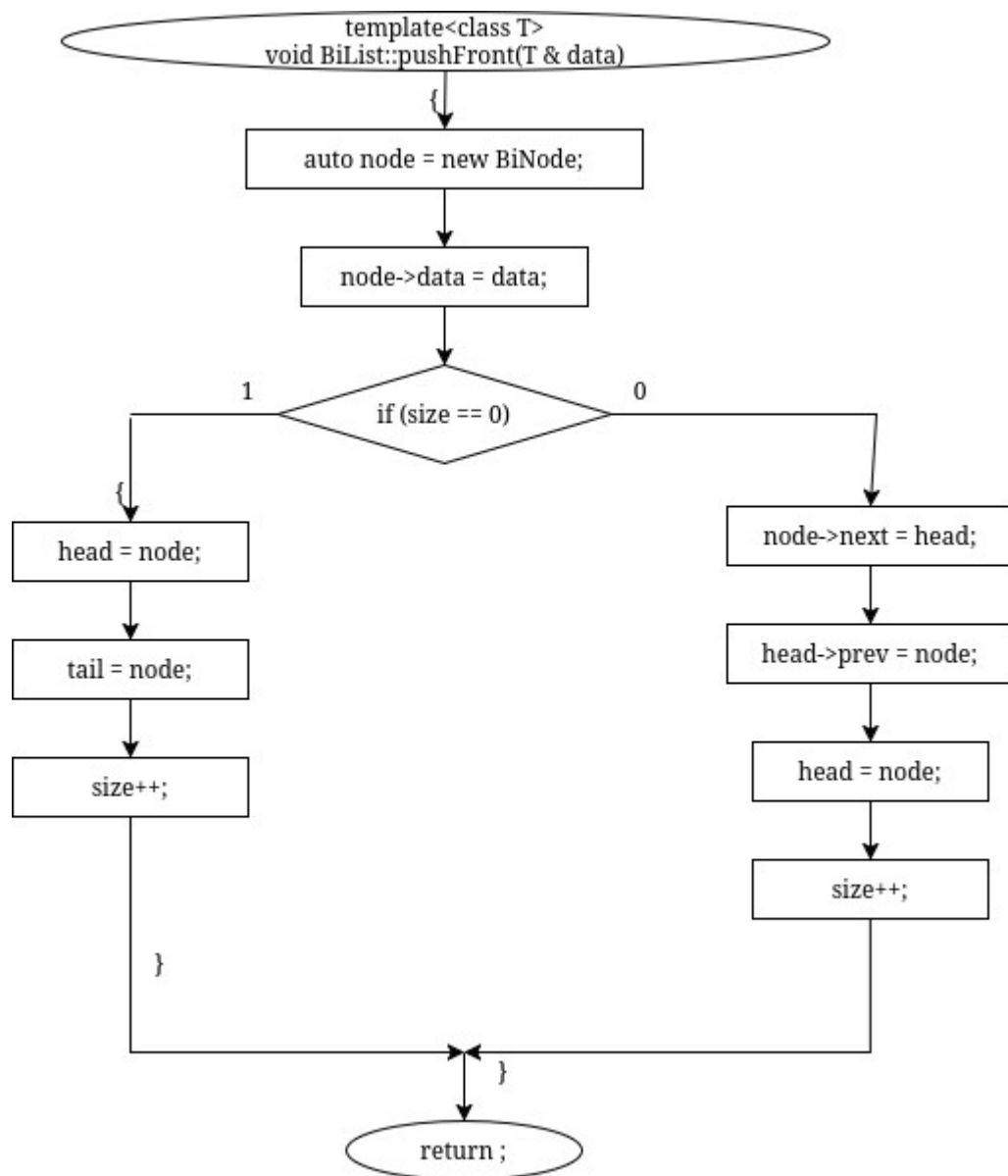


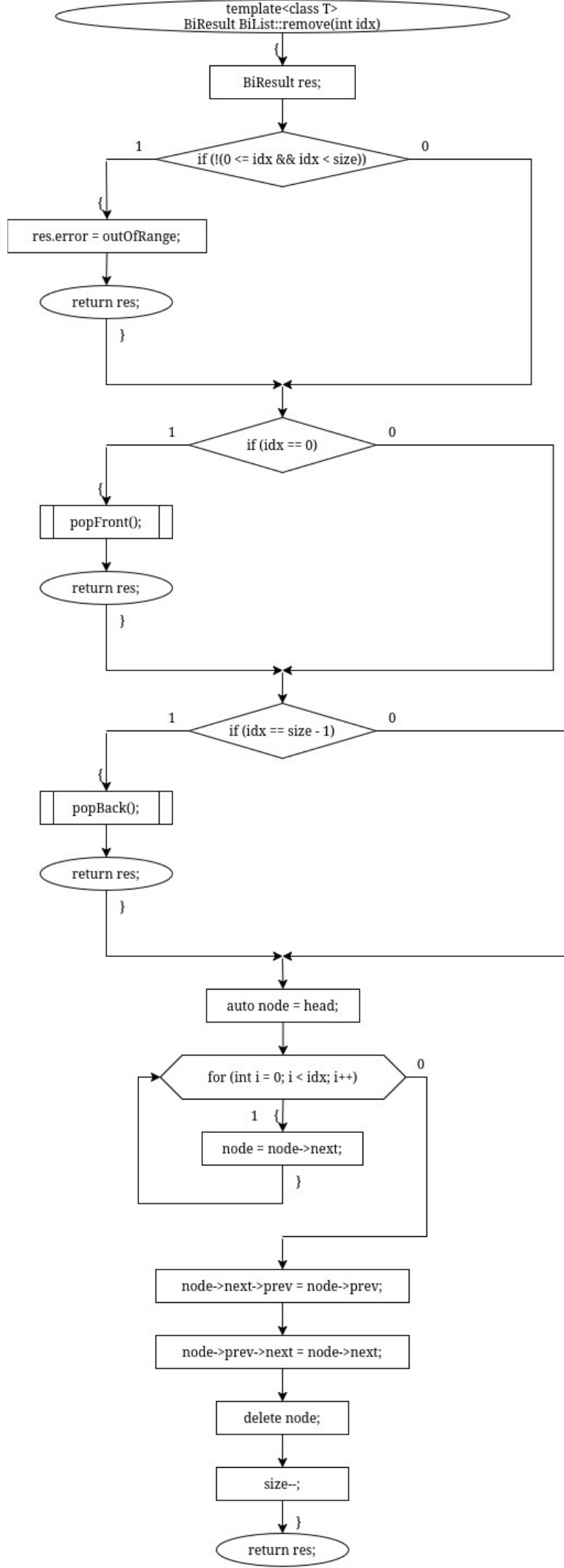












```

#include <iosfwd>
#include "../errors.h"

template<class T>
struct BiNode {
    BiNode* prev = nullptr;
    T data;
    BiNode* next = nullptr;
};

template<class T>
struct BiResult {
    BiNode<T>* node = nullptr;
    int error = ok;
};

template<class T>
struct BiList {
    BiNode<T> * head = nullptr;
    BiNode<T> * tail = nullptr;
    int size = 0;

    void pushBack(T&);
    void popBack();

    void pushFront(T&);
    void popFront();

    BiResult<T> insert(int, T&);
    BiResult<T> remove(int);
    BiResult<T> get(int);
    int find(T&);

    void print(std::ostream&);
    void clear();
};

```

```

template<class T>
void BiList<T>::pushBack(T& data) {
    auto node = new BiNode<T>;
    node->data = data;
    if (size == 0) {
        head = node;
        tail = node;
        size++;
        return;
    }

    node->prev = tail;
    tail->next = node;
    tail = node;
    size++;
}

```

```

template<class T>
void BiList<T>::popBack() {
    if (size == 0)
        return;

    auto del = tail;
    tail = tail->prev;

    if (tail != nullptr)
        tail->next = nullptr;
    else
        head = nullptr;

    delete del;
    size--;
}

```

```

template<class T>
void BiList<T>::pushFront(T & data) {
    auto node = new BiNode<T>;
    node->data = data;
    if (size == 0) {
        head = node;
        tail = node;
        size++;
        return;
    }
}

```

```

    node->next = head;
    head->prev = node;
    head = node;
    size++;
}

```

```

template<class T>
void BiList<T>::popFront() {
    if (size == 0)
        return;

    auto del = head;
    head = head->next;

    if (head != nullptr)
        head->prev = nullptr;
    else
        tail = nullptr;

    delete del;
    size--;
}

```

```

template<class T>
BiResult<T> BiList<T>::insert(int idx, T & data) {
    BiResult<T> res;
    if (idx == 0) {
        pushFront(data);
        return res;
    }
    if (idx == size) {
        pushBack(data);
        return res;
    }
    if (!(0 < idx && idx < size)) {
        res.error = outOfRange;
        return res;
    }

    auto node = head;
    for (int i = 0; i < idx; i++) {
        node = node->next;
    }

    auto newNode = new BiNode<T>;

```

```

newNode->data = data;

node->prev->next = newNode;
newNode->prev = node->prev;

node->prev = newNode;
newNode->next = node;

res.node = newNode;
size++;
return res;
}

template<class T>
BiResult<T> BiList<T>::remove(int idx) {
    BiResult<T> res;
    if (!(0 <= idx && idx < size)) {
        res.error = outOfRange;
        return res;
    }
    if (idx == 0) {
        popFront();
        return res;
    }
    if (idx == size - 1) {
        popBack();
        return res;
    }

    auto node = head;
    for (int i = 0; i < idx; i++) {
        node = node->next;
    }

    node->next->prev = node->prev;
    node->prev->next = node->next;

    delete node;
    size--;
    return res;
}

template<class T>
BiResult<T> BiList<T>::get(int idx) {
    BiResult<T> res;

```

```

    if (!(0 <= idx && idx < size)) {
        res.error = outOfRange;
        return res;
    }

    auto node = head;
    for (int i = 0; i < idx; i++) {
        node = node->next;
    }

    res.node = node;
    return res;
}

template<class T>
int BiList<T>::find(T & data) {
    auto node = head;
    int i = 0;
    while (node != nullptr) {
        if (node->data == data) {
            return i;
        }
        i++;
        node = node->next;
    }

    return -1;
}

template<class T>
void BiList<T>::print(std::ostream& stream) {
    if (size == 0) {
        stream << "empty\n";
        return;
    }
    auto node = head;
    while (node != nullptr) {
        stream << node->data << ' ';
        node = node->next;
    }
    stream << '\n';
}

template<class T>
void BiList<T>::clear() {

```

```

    auto node = head;
    while (node != nullptr) {
        auto tmp = node->next;
        delete node;
        node = tmp;
    }
    head = nullptr;
    tail = nullptr;
    size = 0;
}

```

```

int mainBi() {
    using namespace std;
    int n, m, k;
    BiList<string> list;
    cout << "Size:\n";
    cin >> n;
    for (int i = 0; i < n; i++) {
        string s;
        cin >> s;
        list.pushBack(s);
    }
    list.print(cout);

    cout << "Delete:\n";
    cin >> m;
    list.remove(m - 1);
    list.print(cout);
    cout << "Key:\n";
    string key;
    cin >> key;
    int idx = list.find(key);
    cout << "Index: " << idx << '\n';
    cout << "Amount:\n";
    cin >> k;

    cout << "First " << k << " words:\n";
    for (int i = 0; i < k; i++) {
        string s;
        cin >> s;
        list.insert(idx, s);
        idx++;
        list.print(cout);
    }
}

```

```

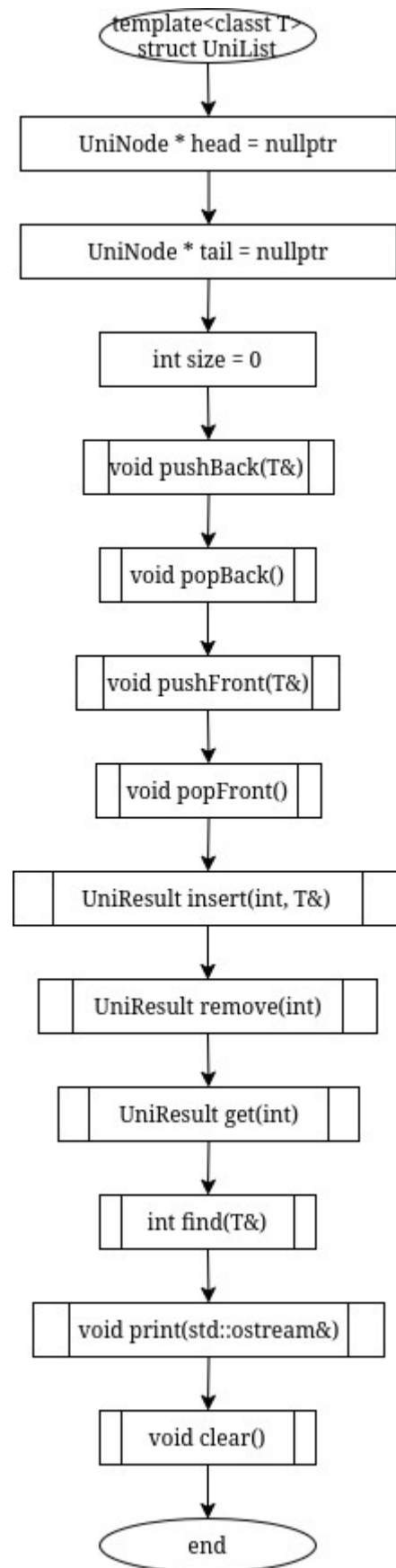
idx++;
cout << "Second " << k << " words:\n";
for (int i = 0; i < k; i++) {
    string s;
    cin >> s;
    list.insert(idx, s);
    idx++;
    list.print(cout);
}

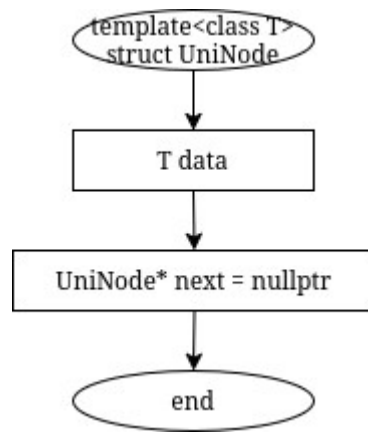
fstream file("file.txt", ios::in | ios::out);
list.print(file);
file.seekp(0);

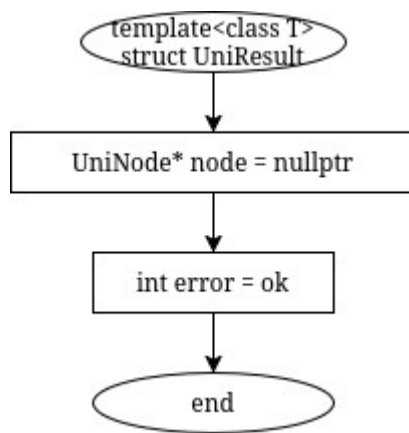
list.clear();
list.print(cout);
for (int i = 0; i < n - 1 + 2 * k; i++) {
    string s;
    file >> s;
    list.pushBack(s);
}
file.close();
list.print(cout);
list.clear();

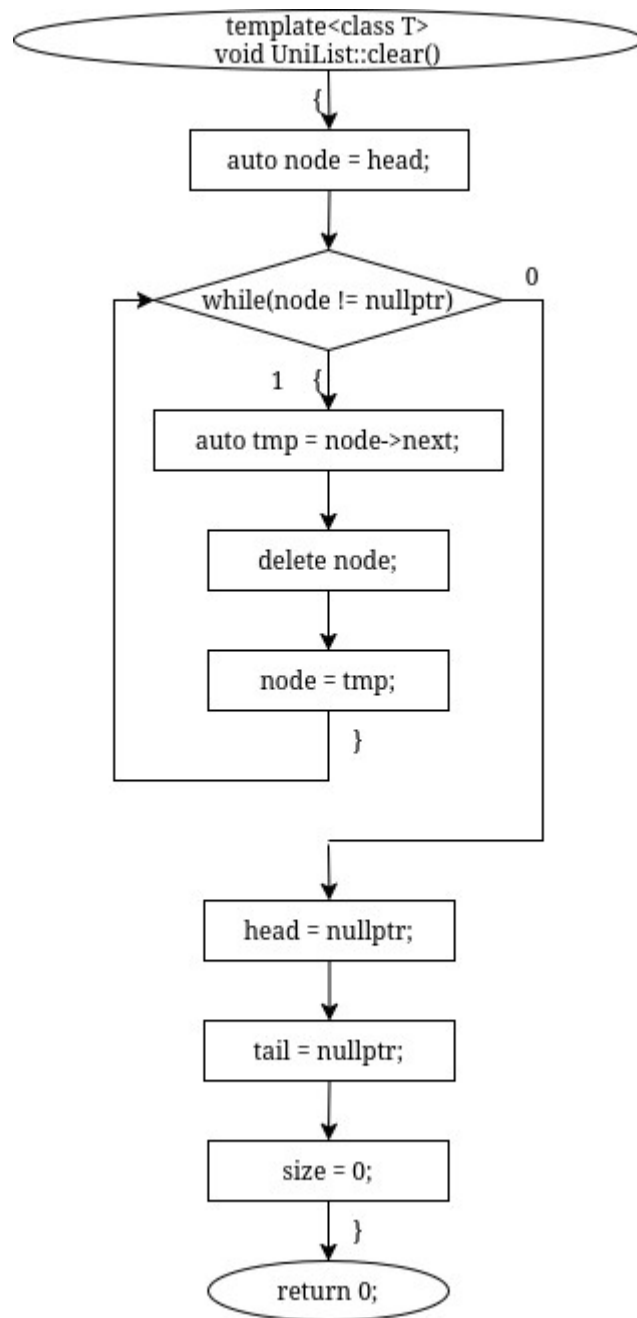
return 0;
}

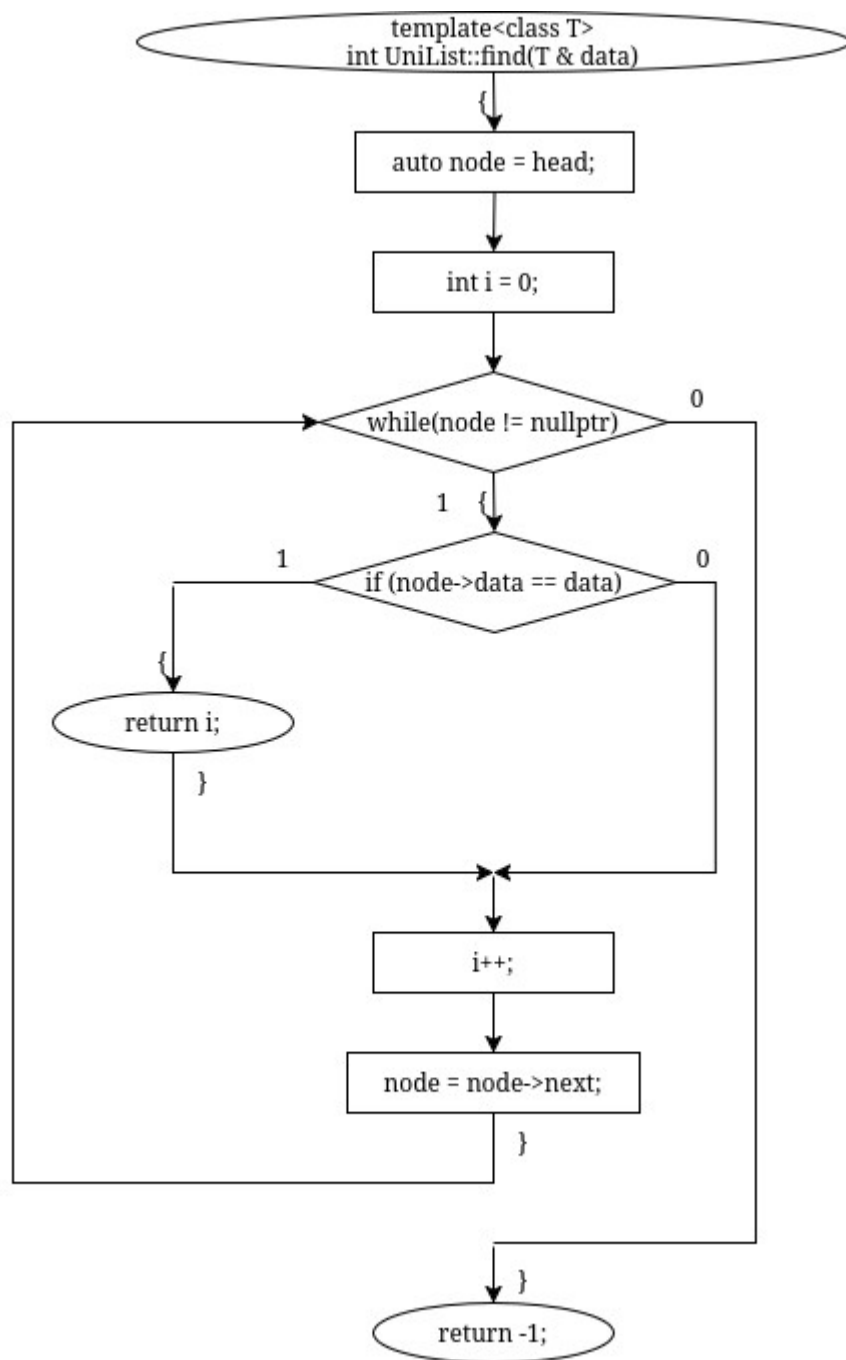
```

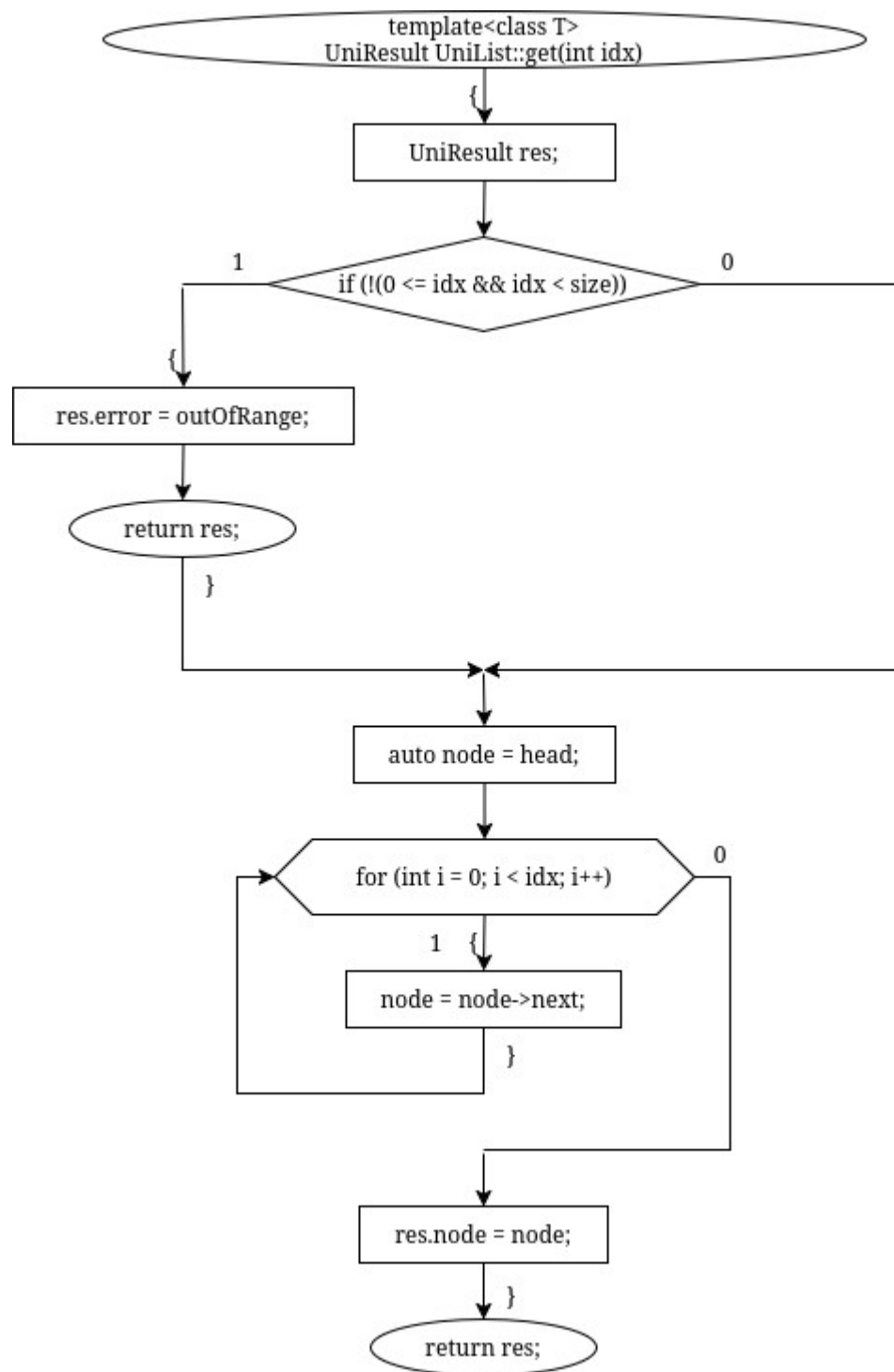


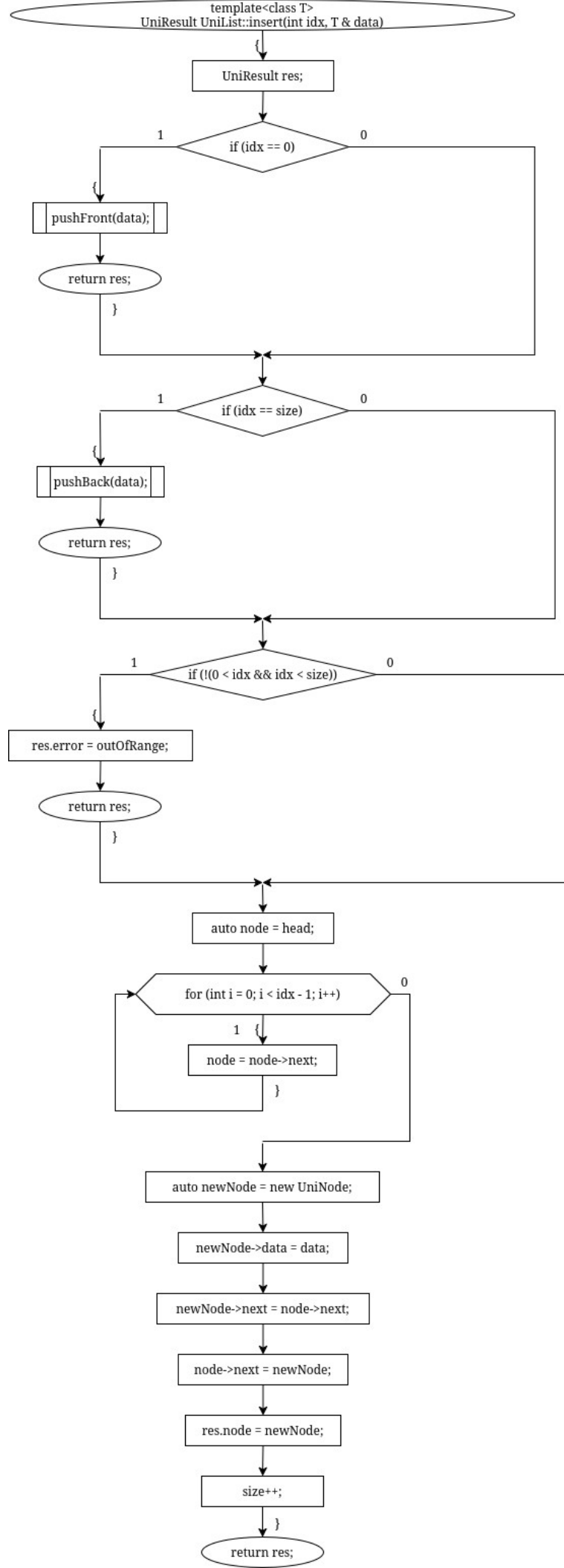


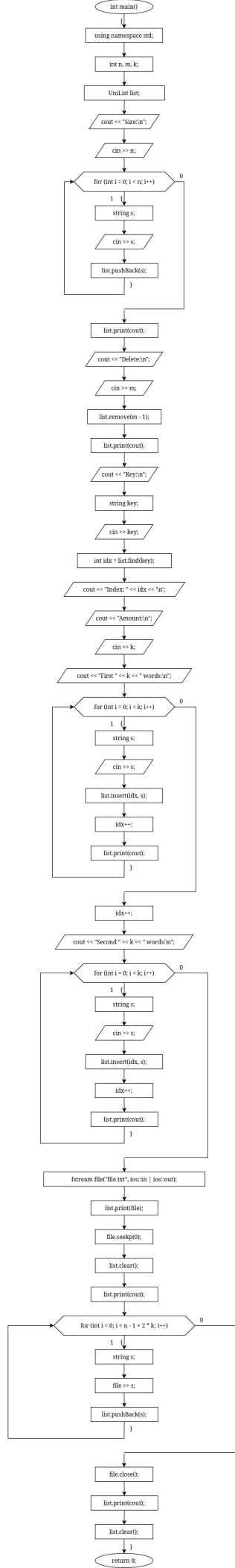


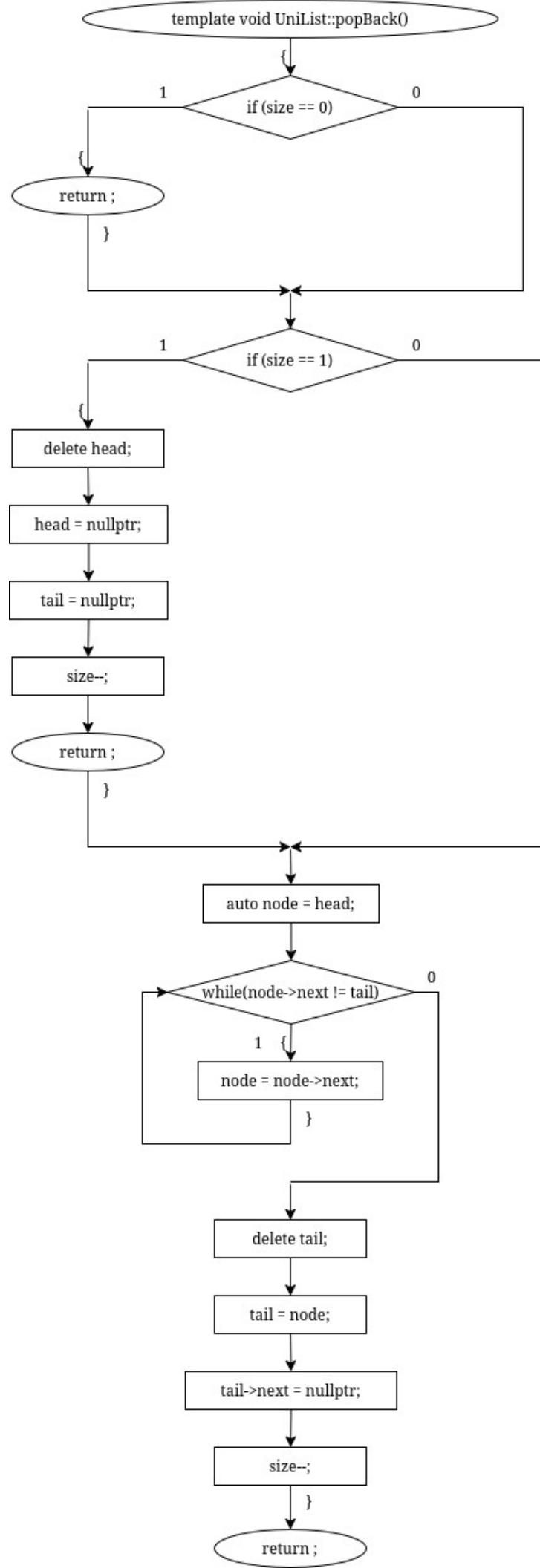


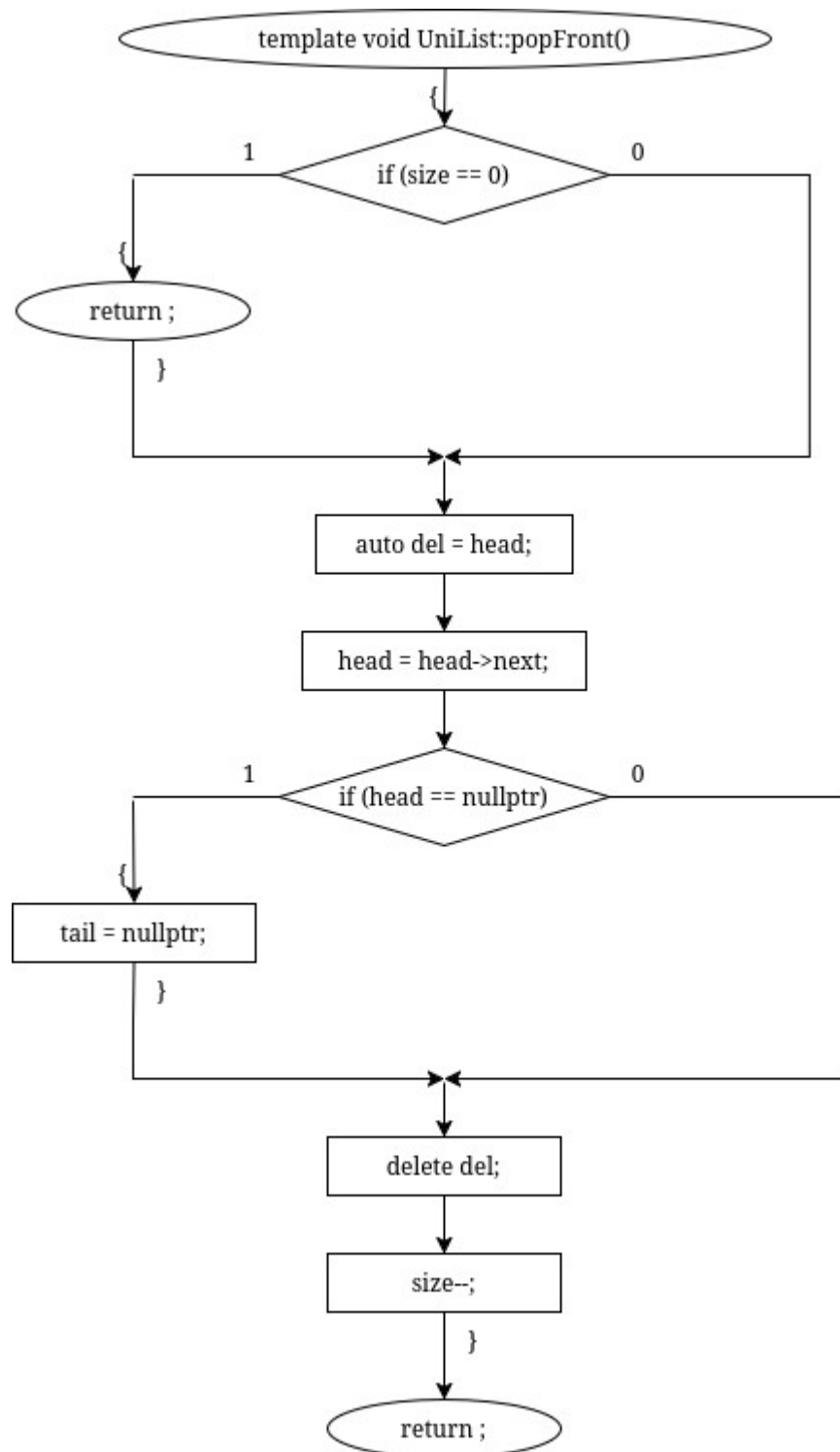


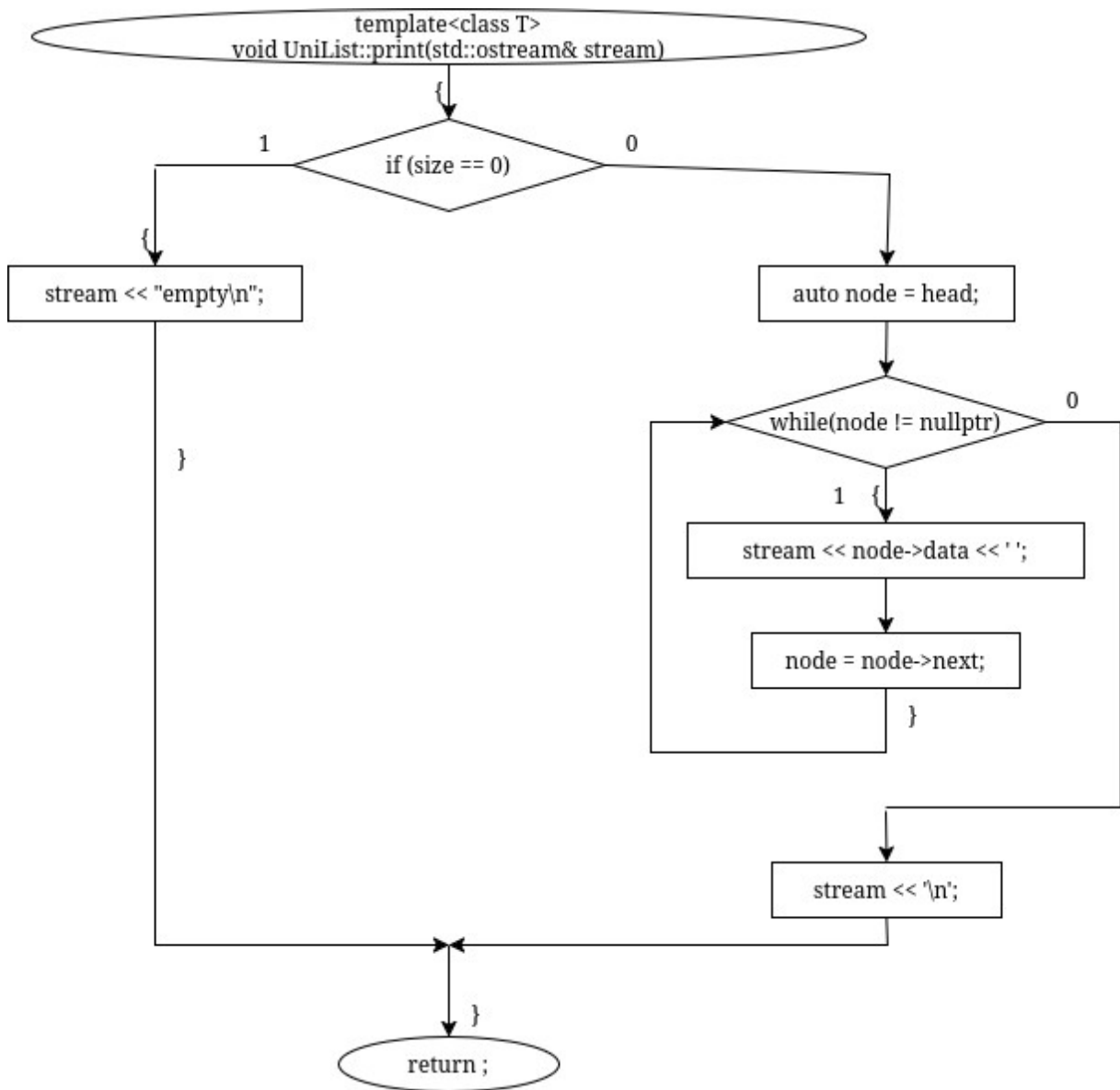


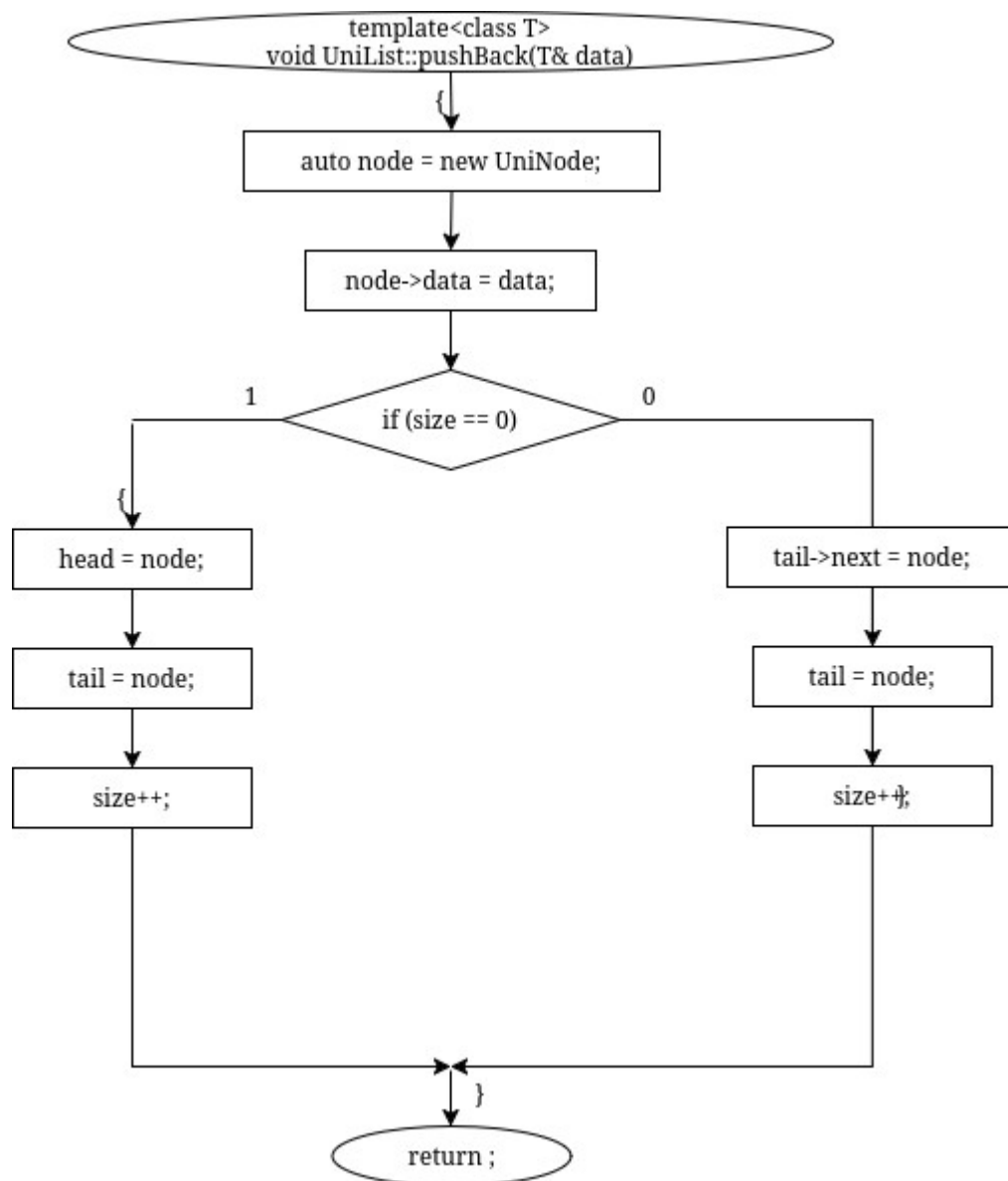


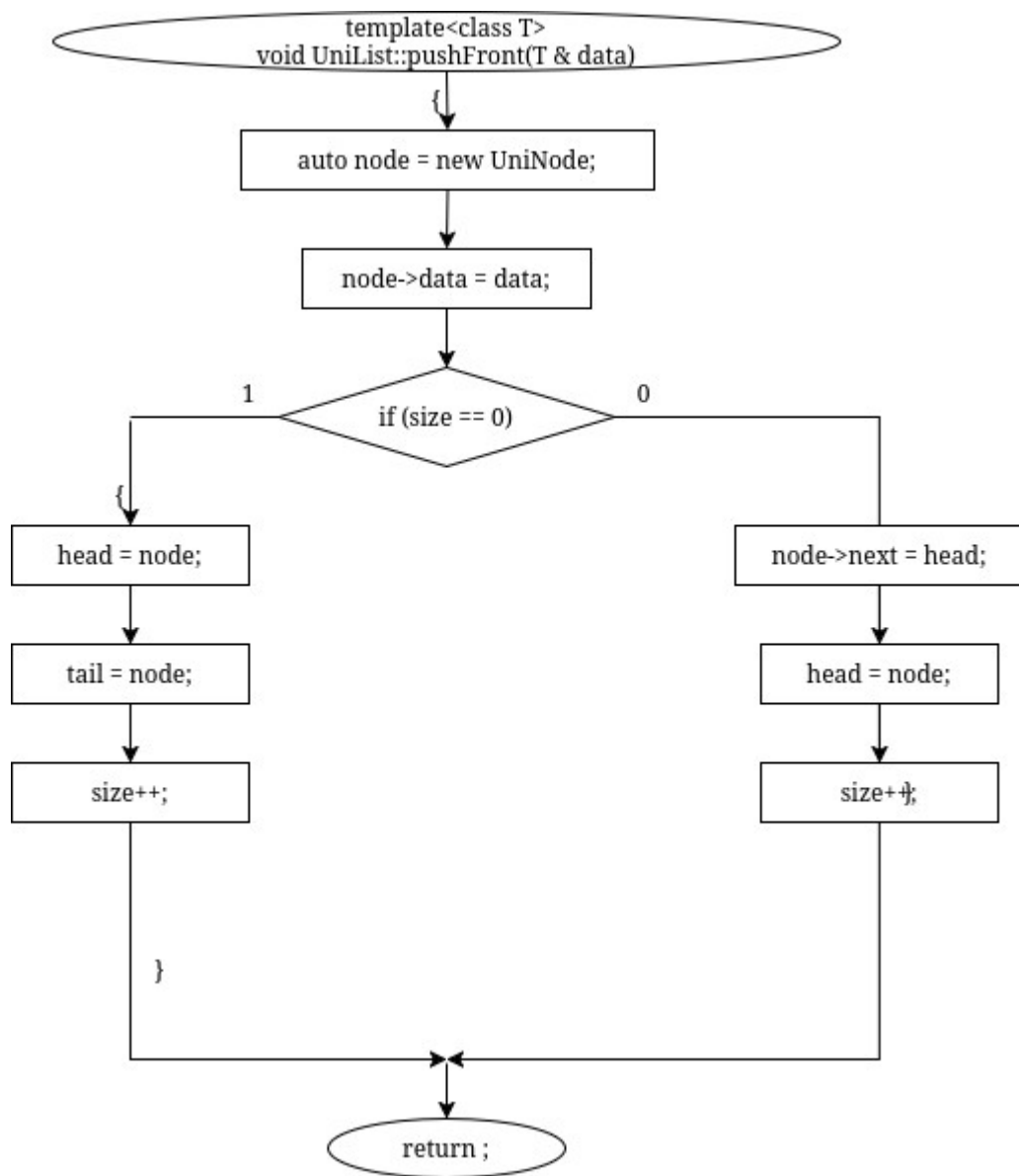


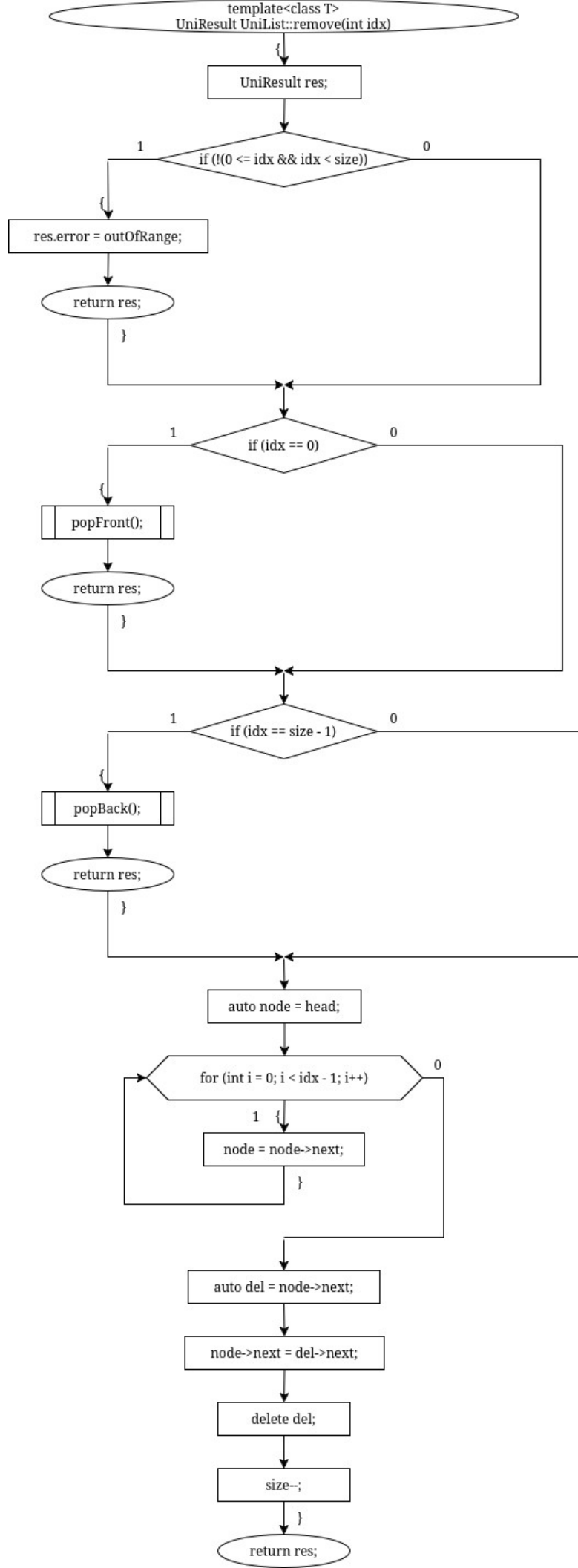












```
#include <iosfwd>
#include "../errors.h"
```

```
template<class T>
struct UniNode {
    T data;
    UniNode* next = nullptr;
};
```

```
template<class T>
struct UniResult {
    UniNode<T>* node = nullptr;
    int error = ok;
};
```

```
template<class T>
struct UniList {
    UniNode<T> * head = nullptr;
    UniNode<T> * tail = nullptr;
    int size = 0;
```

```
    void pushBack(T&);
    void popBack();
```

```
    void pushFront(T&);
    void popFront();
```

```
    UniResult<T> insert(int, T&);
    UniResult<T> remove(int);
    UniResult<T> get(int);
    int find(T&);
```

```
    void print(std::ostream&);
    void clear();
```

```
};
```

```

template<class T>
void UniList<T>::pushBack(T& data) {
    auto node = new UniNode<T>;
    node->data = data;
    if (size == 0) {
        head = node;
        tail = node;
        size++;
        return;
    }

    tail->next = node;
    tail = node;
    size++;
}

```

```

template<class T>
void UniList<T>::popBack() {
    if (size == 0)
        return;

    if (size == 1) {
        delete head;
        head = nullptr;
        tail = nullptr;
        size--;
        return;
    }

    auto node = head;
    while (node->next != tail)
        node = node->next;

    delete tail;
    tail = node;
    tail->next = nullptr;
    size--;
}

```

```

template<class T>
void UniList<T>::pushFront(T & data) {
    auto node = new UniNode<T>;
    node->data = data;
    if (size == 0) {

```

```

        head = node;
        tail = node;
        size++;
        return;
    }

    node->next = head;
    head = node;
    size++;
}

template<class T>
void UniList<T>::popFront() {
    if (size == 0)
        return;

    auto del = head;
    head = head->next;

    if (head == nullptr)
        tail = nullptr;

    delete del;
    size--;
}

template<class T>
UniResult<T> UniList<T>::insert(int idx, T & data) {
    UniResult<T> res;
    if (idx == 0) {
        pushFront(data);
        return res;
    }
    if (idx == size) {
        pushBack(data);
        return res;
    }
    if (!(0 < idx && idx < size)) {
        res.error = outOfRange;
        return res;
    }

    auto node = head;
    for (int i = 0; i < idx - 1; i++)
        node = node->next;

```

```

    auto newNode = new UniNode<T>;
    newNode->data = data;
    newNode->next = node->next;
    node->next = newNode;

    res.node = newNode;
    size++;
    return res;
}

template<class T>
UniResult<T> UniList<T>::remove(int idx) {
    UniResult<T> res;
    if (!(0 <= idx && idx < size)) {
        res.error = outOfRange;
        return res;
    }
    if (idx == 0) {
        popFront();
        return res;
    }
    if (idx == size - 1) {
        popBack();
        return res;
    }

    auto node = head;
    for (int i = 0; i < idx - 1; i++)
        node = node->next;

    auto del = node->next;
    node->next = del->next;
    delete del;
    size--;
    return res;
}

template<class T>
UniResult<T> UniList<T>::get(int idx) {
    UniResult<T> res;
    if (!(0 <= idx && idx < size)) {
        res.error = outOfRange;
        return res;
    }
}

```

```

    auto node = head;
    for (int i = 0; i < idx; i++) {
        node = node->next;
    }

```

```

    res.node = node;
    return res;
}

```

```

template<class T>
int UniList<T>::find(T & data) {
    auto node = head;
    int i = 0;
    while (node != nullptr) {
        if (node->data == data) {
            return i;
        }
        i++;
        node = node->next;
    }

```

```

    return -1;
}

```

```

template<class T>
void UniList<T>::print(std::ostream& stream) {
    if (size == 0) {
        stream << "empty\n";
        return;
    }
    auto node = head;
    while (node != nullptr) {
        stream << node->data << ' ';
        node = node->next;
    }
    stream << '\n';
}

```

```

template<class T>
void UniList<T>::clear() {
    auto node = head;
    while (node != nullptr) {
        auto tmp = node->next;
        delete node;

```

```

        node = tmp;
    }
    head = nullptr;
    tail = nullptr;
    size = 0;
}

```

```

int mainUni() {
    using namespace std;
    int n, m, k;
    UniList<string> list;
    cout << "Size:\n";
    cin >> n;
    for (int i = 0; i < n; i++) {
        string s;
        cin >> s;
        list.pushBack(s);
    }
    list.print(cout);

    cout << "Delete:\n";
    cin >> m;
    list.remove(m - 1);
    list.print(cout);
    cout << "Key:\n";
    string key;
    cin >> key;
    int idx = list.find(key);
    cout << "Index: " << idx << '\n';
    cout << "Amount:\n";
    cin >> k;

    cout << "First " << k << " words:\n";
    for (int i = 0; i < k; i++) {
        string s;
        cin >> s;
        list.insert(idx, s);
        idx++;
        list.print(cout);
    }

    idx++;
    cout << "Second " << k << " words:\n";
    for (int i = 0; i < k; i++) {

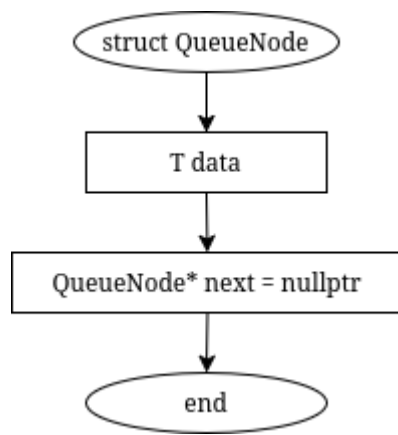
```

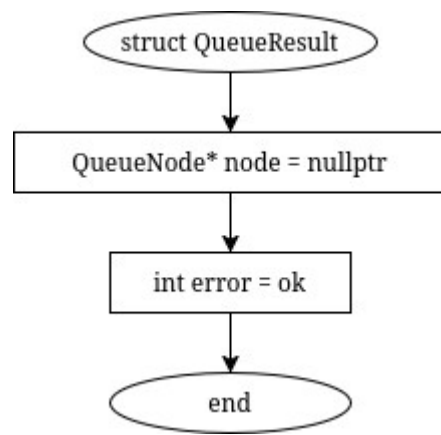
```
    string s;
    cin >> s;
    list.insert(idx, s);
    idx++;
    list.print(cout);
}

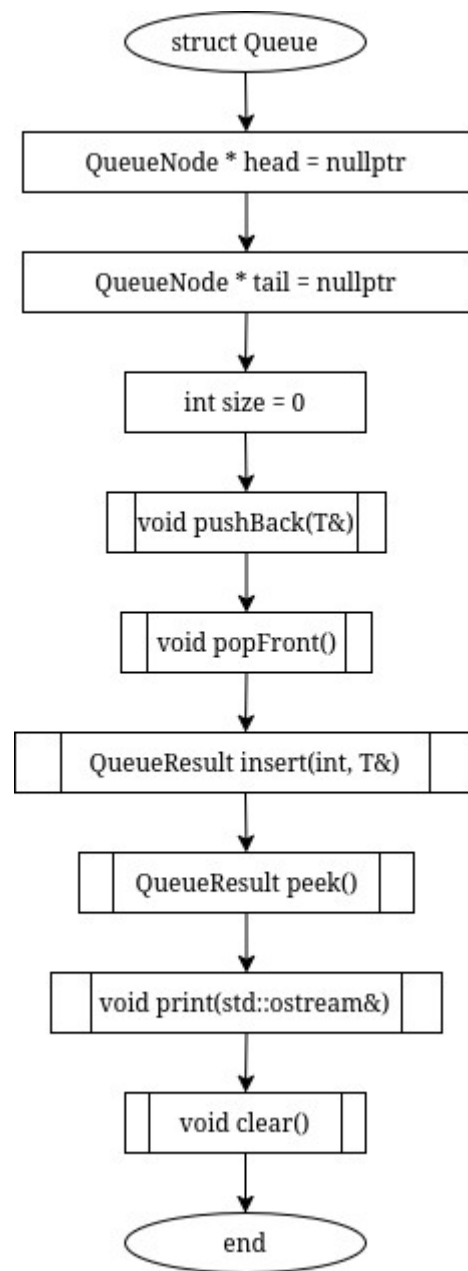
fstream file("file.txt", ios::in | ios::out);
list.print(file);
file.seekp(0);

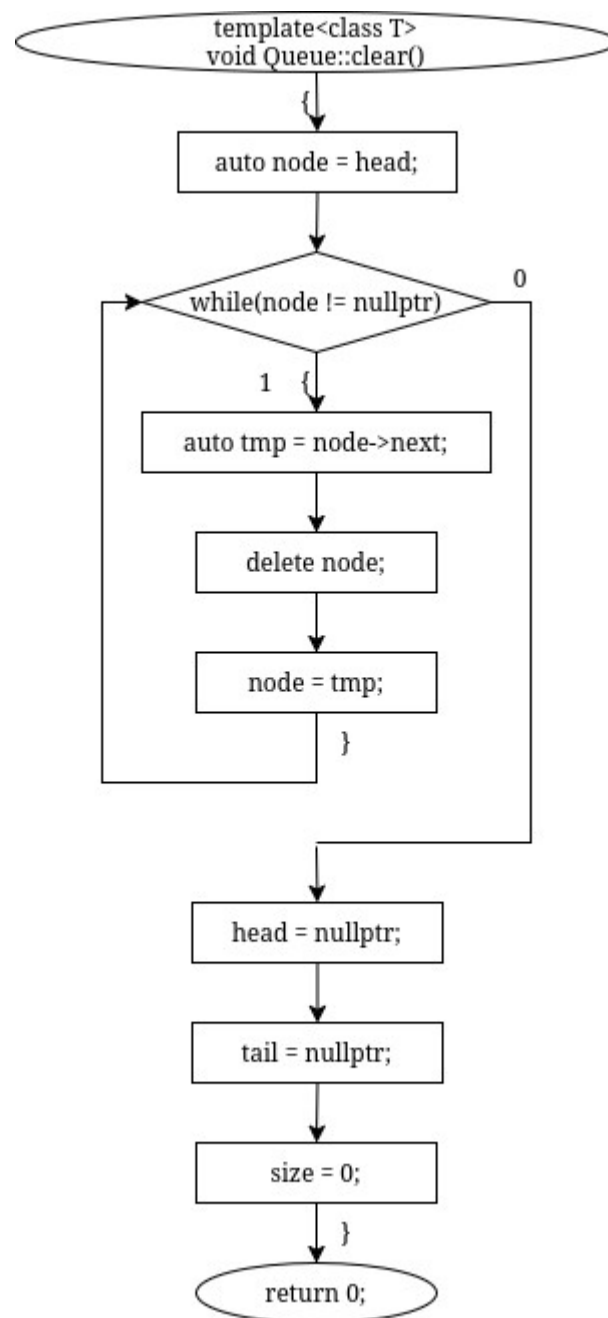
list.clear();
list.print(cout);
for (int i = 0; i < n - 1 + 2 * k; i++) {
    string s;
    file >> s;
    list.pushBack(s);
}
file.close();
list.print(cout);
list.clear();

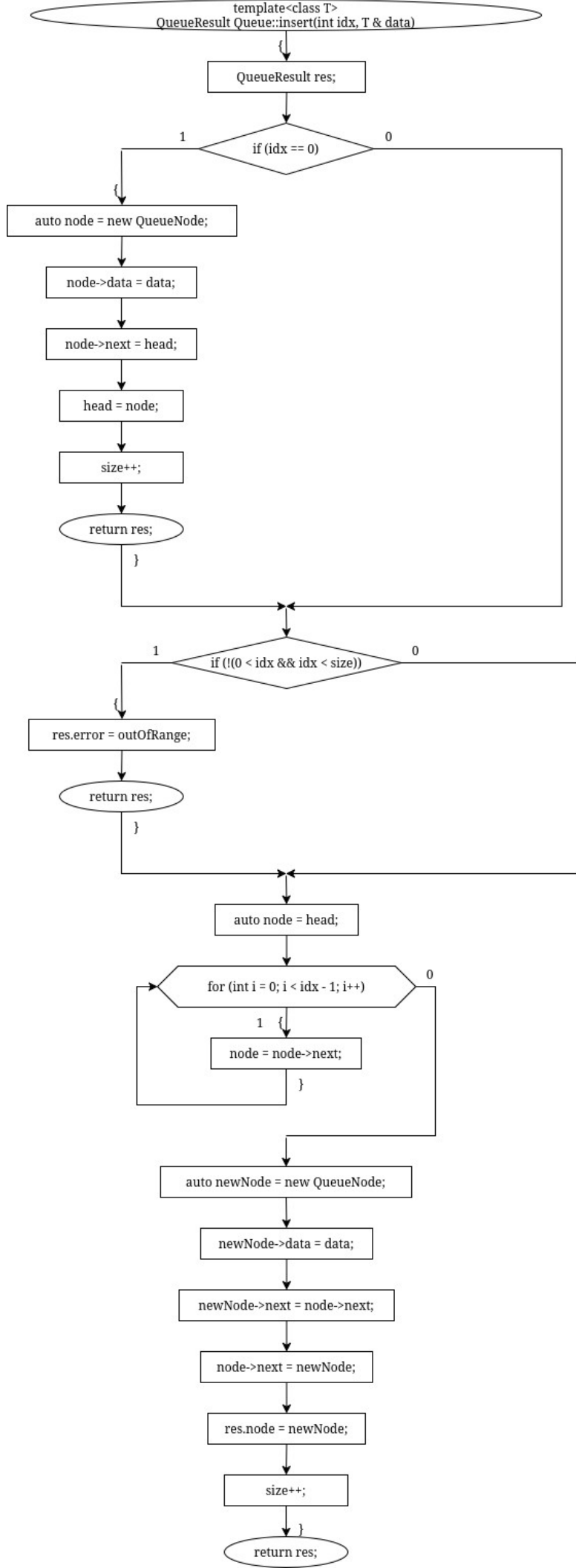
return 0;
}
```

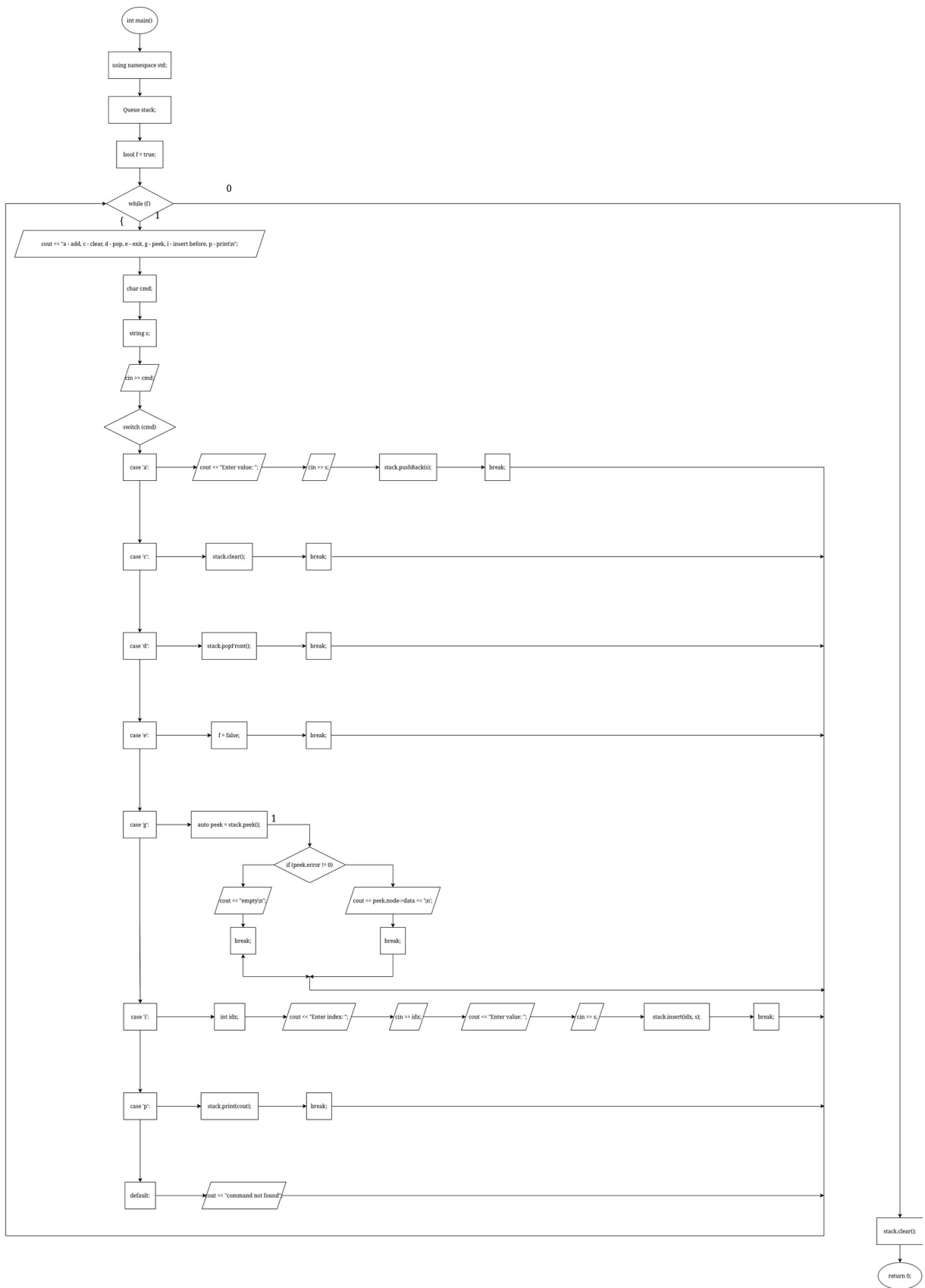


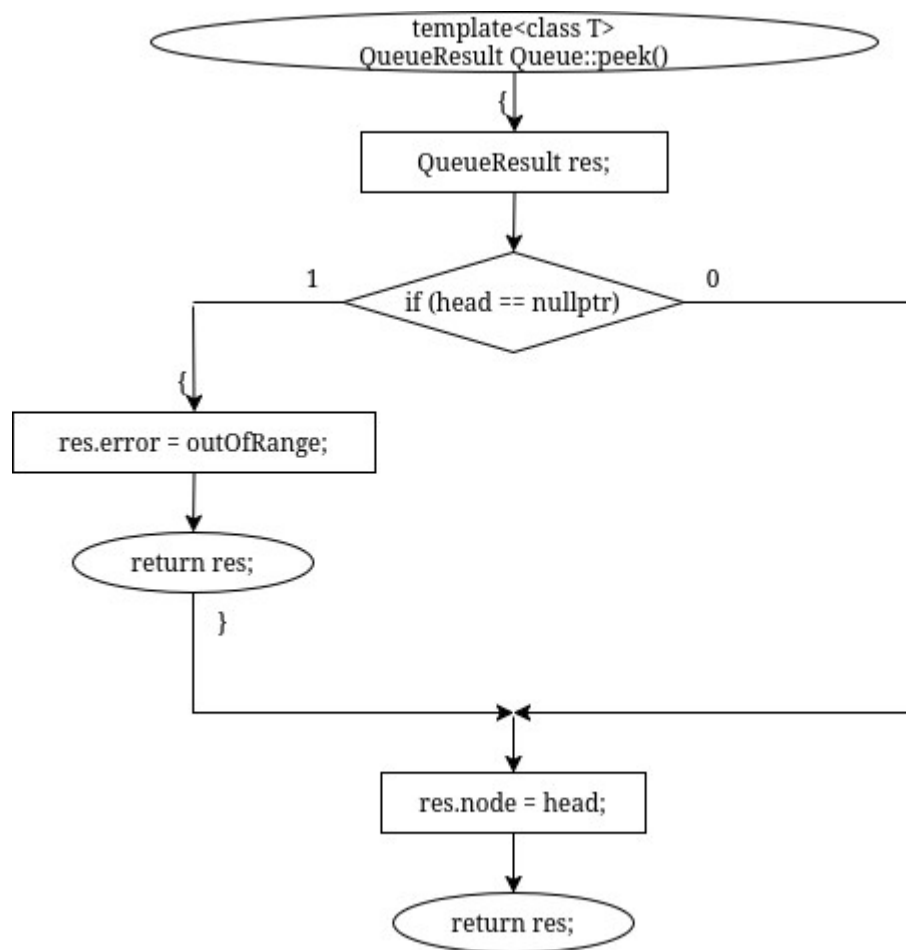


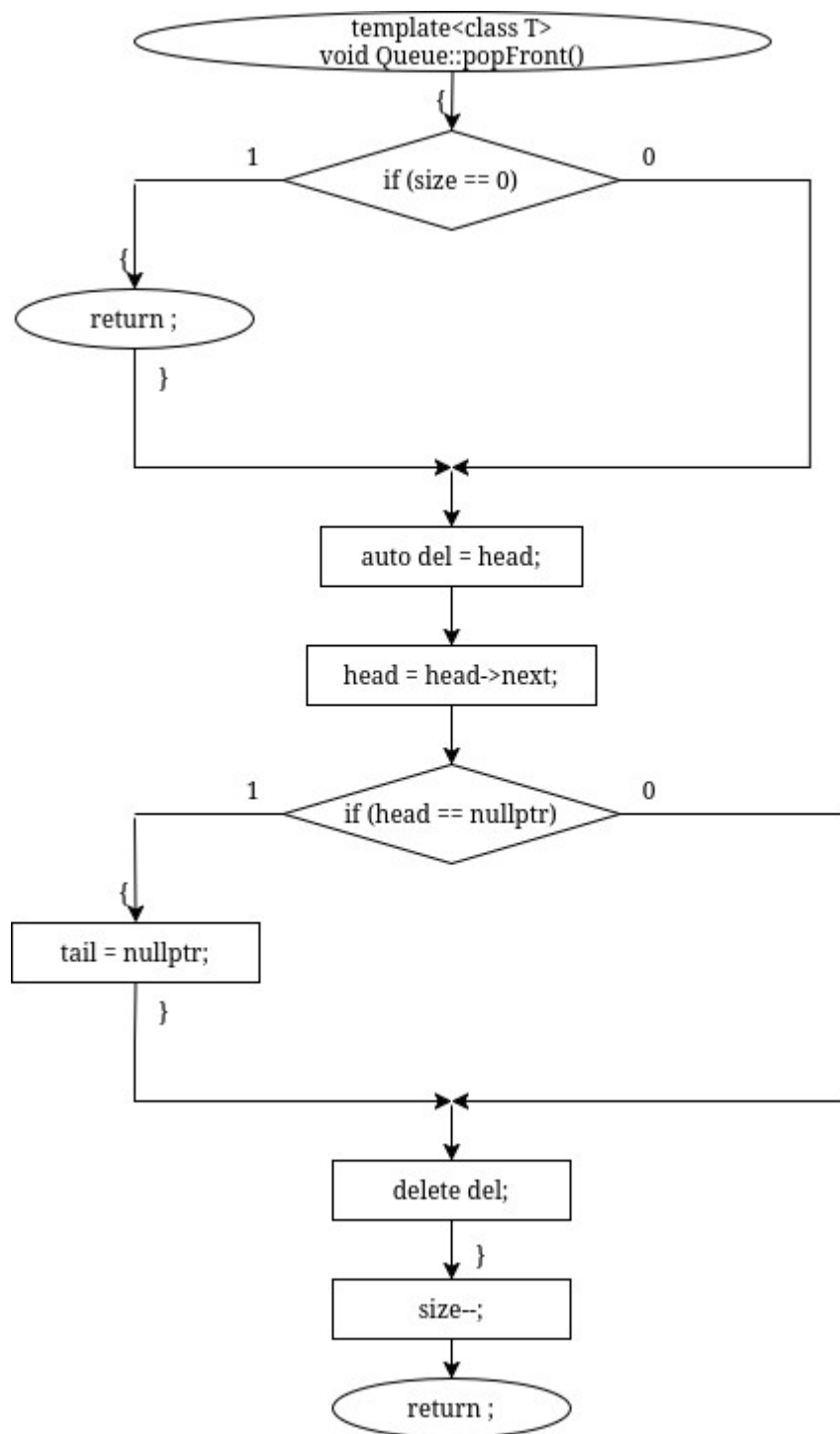


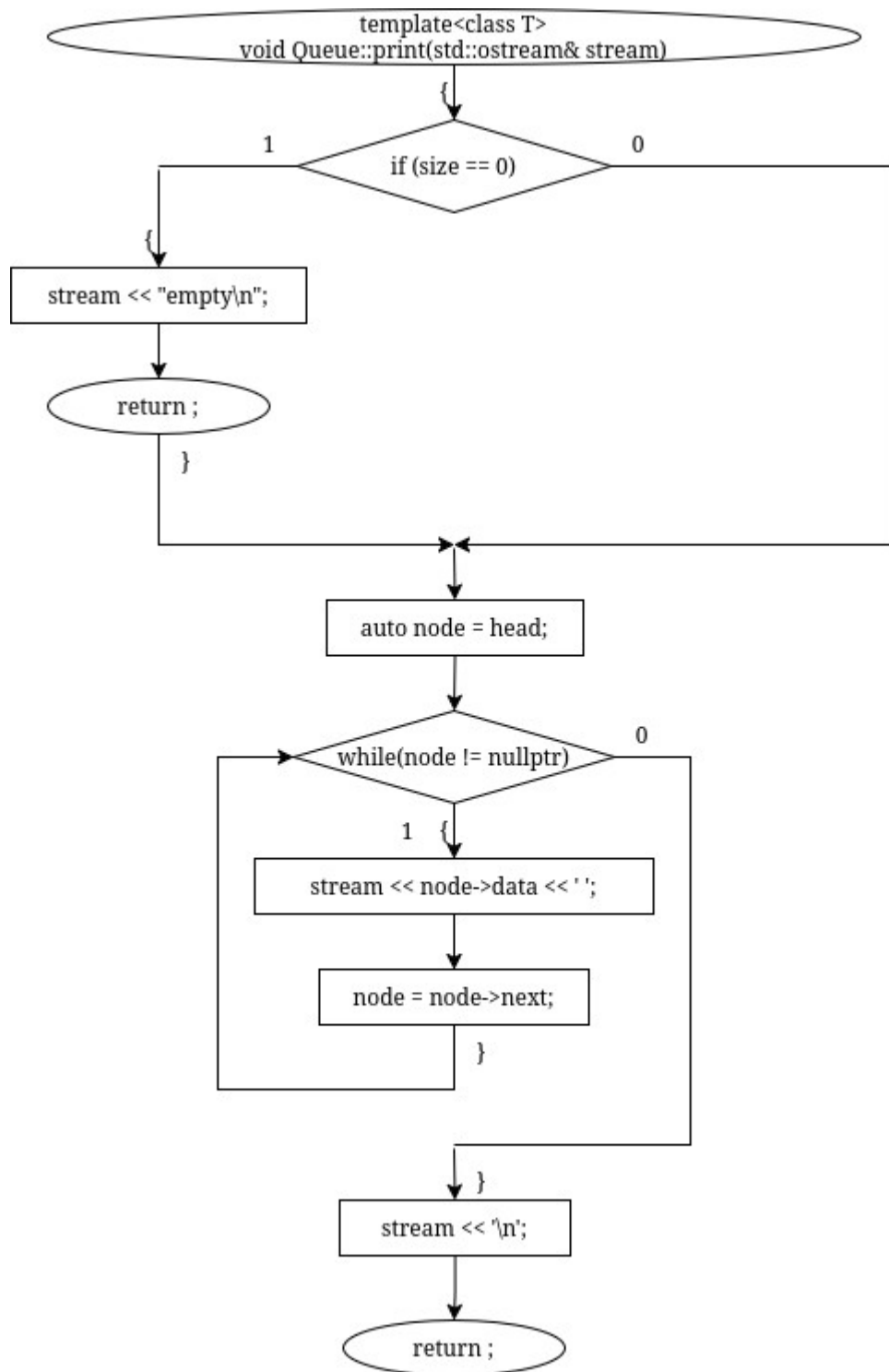


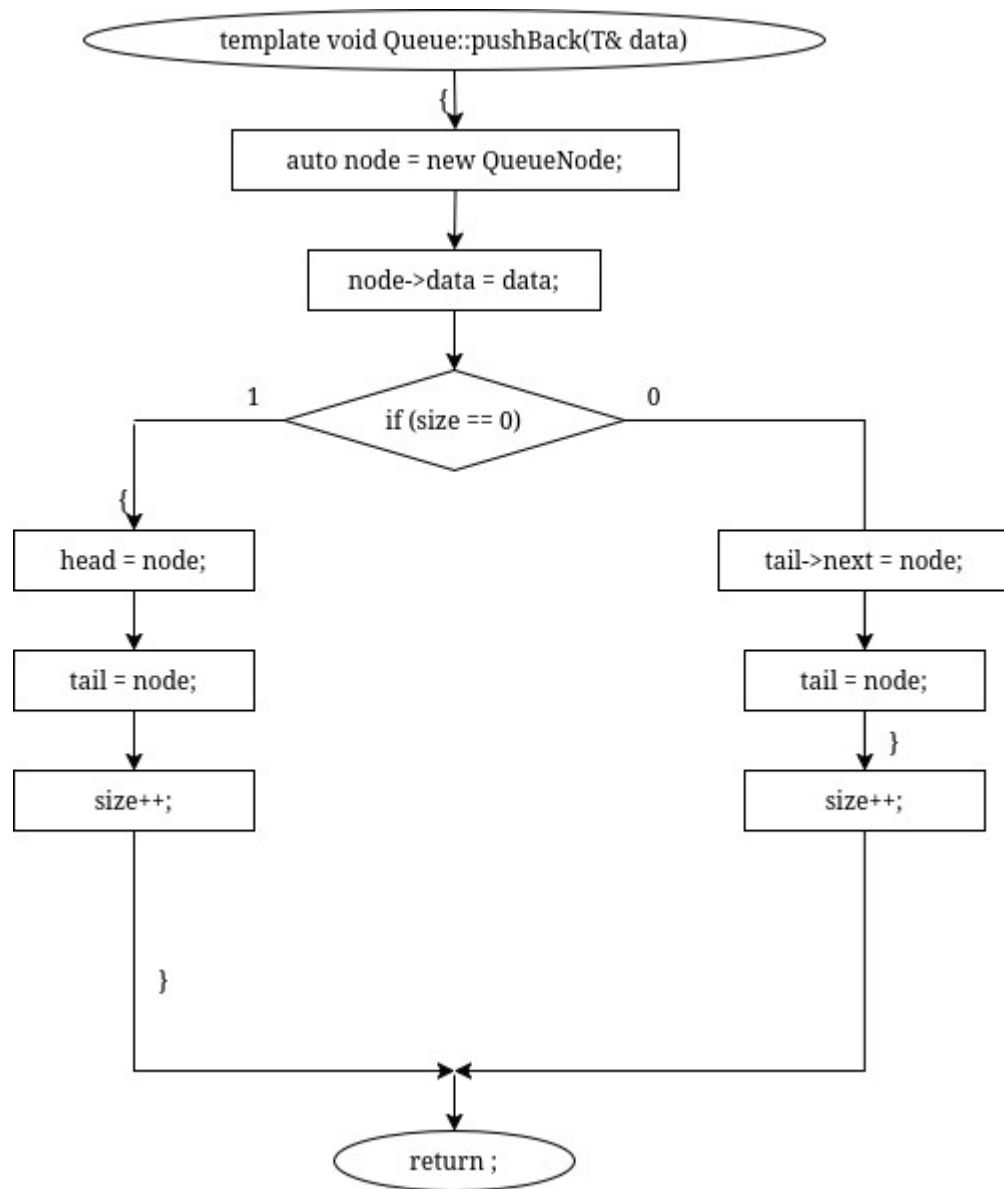












```
#include <iosfwd>
#include "../errors.h"
```

```
template<class T>
struct QueueNode {
    T data;
    QueueNode* next = nullptr;
};
```

```
template<class T>
struct QueueResult {
    QueueNode<T>* node = nullptr;
    int error = ok;
};
```

```
template<class T>
struct Queue {
    QueueNode<T> * head = nullptr;
    QueueNode<T> * tail = nullptr;
    int size = 0;

    void pushBack(T&);
    void popFront();
    QueueResult<T> insert(int, T&);
    QueueResult<T> peek();

    void print(std::ostream&);
    void clear();
};
```

```
template<class T>
void Queue<T>::pushBack(T& data) {
    auto node = new QueueNode<T>;
    node->data = data;
    if (size == 0) {
        head = node;
        tail = node;
        size++;
        return;
    }

    tail->next = node;
```

```
    tail = node;
    size++;
}
```

```
template<class T>
void Queue<T>::popFront() {
    if (size == 0)
        return;

    auto del = head;
    head = head->next;

    if (head == nullptr)
        tail = nullptr;

    delete del;
    size--;
}
```

```
template<class T>
QueueResult<T> Queue<T>::insert(int idx, T & data) {
    QueueResult<T> res;
    if (idx == 0) {
        auto node = new QueueNode<T>;
        node->data = data;
        node->next = head;
        head = node;
        size++;
        return res;
    }

    if (!(0 < idx && idx < size)) {
        res.error = outOfRange;
        return res;
    }

    auto node = head;
    for (int i = 0; i < idx - 1; i++)
        node = node->next;

    auto newNode = new QueueNode<T>;
    newNode->data = data;
    newNode->next = node->next;
    node->next = newNode;
}
```

```

    res.node = newNode;
    size++;
    return res;
}

```

```

template<class T>
QueueResult<T> Queue<T>::peek() {
    QueueResult<T> res;
    if (head == nullptr) {
        res.error = outOfRange;
        return res;
    }
    res.node = head;
    return res;
}

```

```

template<class T>
void Queue<T>::print(std::ostream& stream) {
    if (size == 0) {
        stream << "empty\n";
        return;
    }
    auto node = head;
    while (node != nullptr) {
        stream << node->data << ' ';
        node = node->next;
    }
    stream << '\n';
}

```

```

template<class T>
void Queue<T>::clear() {
    auto node = head;
    while (node != nullptr) {
        auto tmp = node->next;
        delete node;
        node = tmp;
    }
    head = nullptr;
    tail = nullptr;
    size = 0;
}

```

```

int main() {

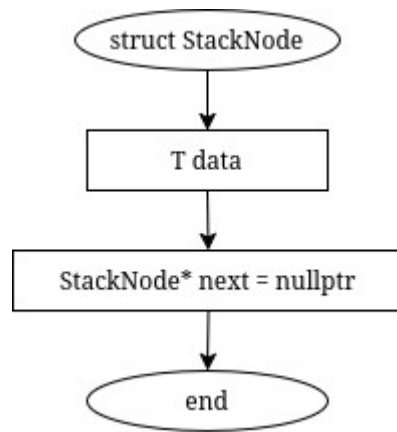
```

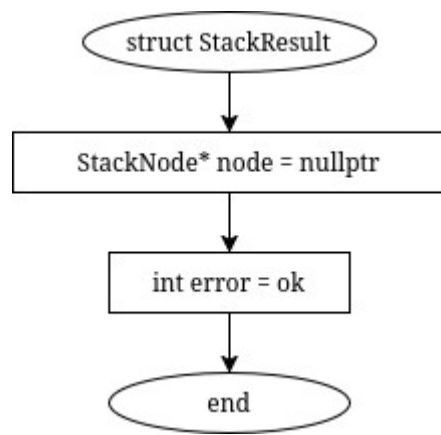
```

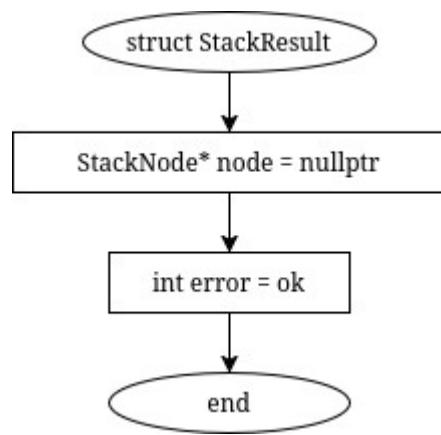
using namespace std;
Queue<string> stack;
bool f = true;
while (f) {
    cout << "a - add, c - clear, d - pop, e - exit, g - peek, i - insert before, p - print\n";
    char cmd;
    string s;
    cin >> cmd;
    switch (cmd) {
        case 'a':
            cout << "Enter value: ";
            cin >> s;
            stack.pushBack(s);
            break;
        case 'c':
            stack.clear();
            break;
        case 'd':
            stack.popFront();
            break;
        case 'e':
            f = false;
            break;
        case 'g': {
            auto peek = stack.peek();
            if (peek.error != 0) {
                cout << "empty\n";
                break;
            }
            cout << peek.node->data << '\n';
            break;
        }
        case 'i':
            int idx;
            cout << "Enter index: ";
            cin >> idx;
            cout << "Enter value: ";
            cin >> s;
            stack.insert(idx, s);
            break;
        case 'p':
            stack.print(cout);
            break;
        default:
            cout << "command not found";
    }
}

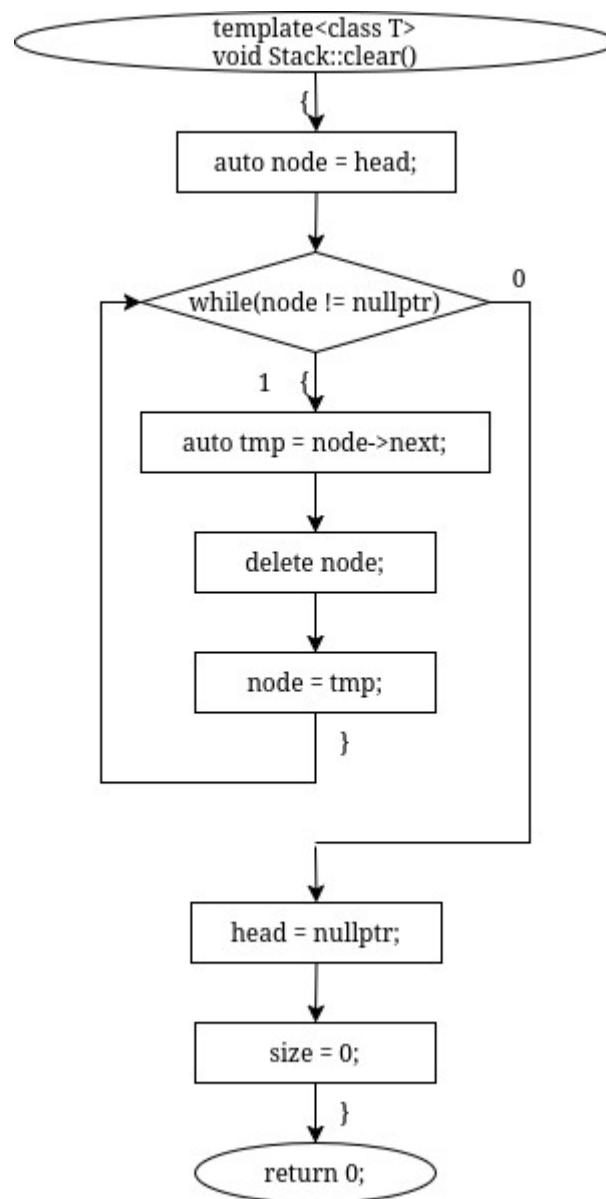
```

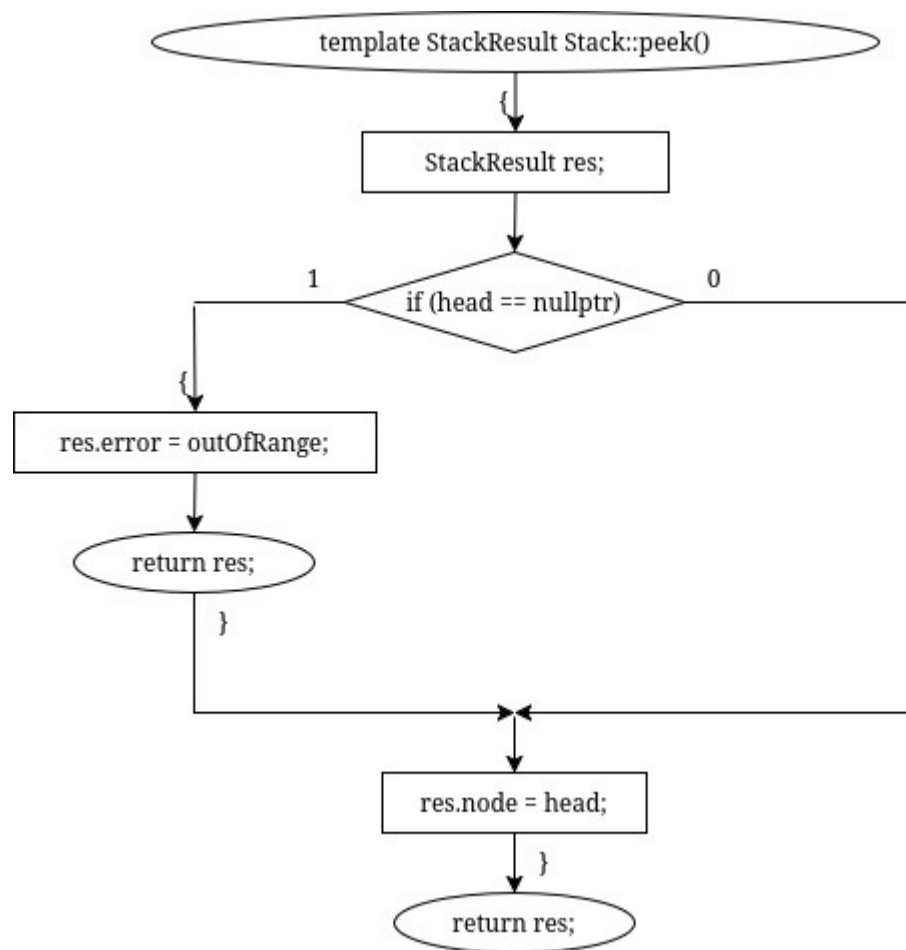
```
    }  
}  
stack.clear();  
return 0;  
}
```

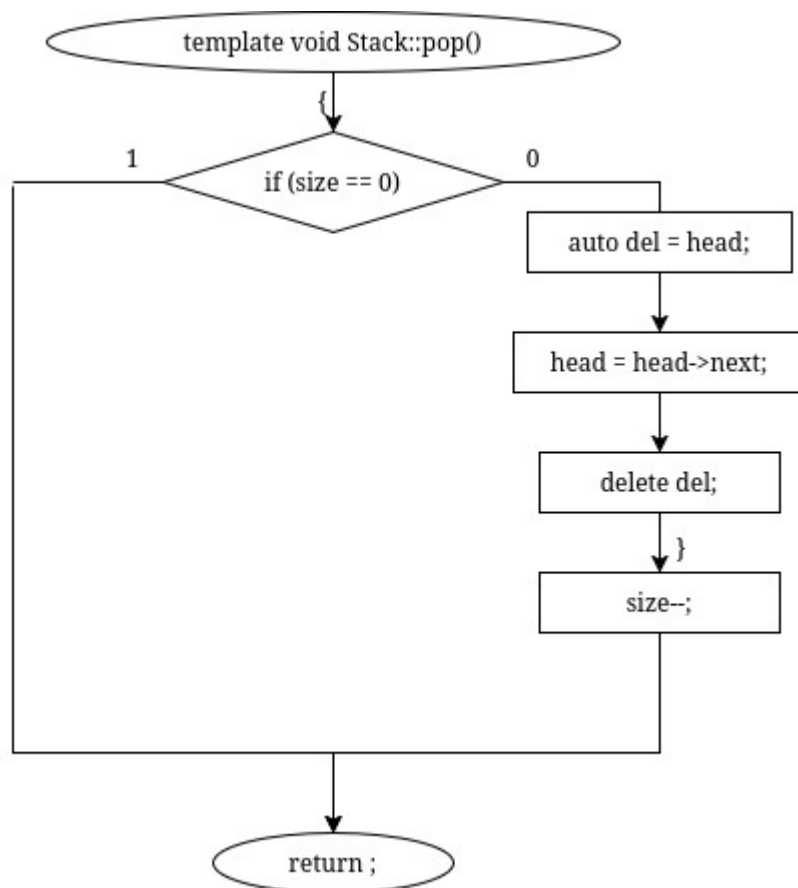


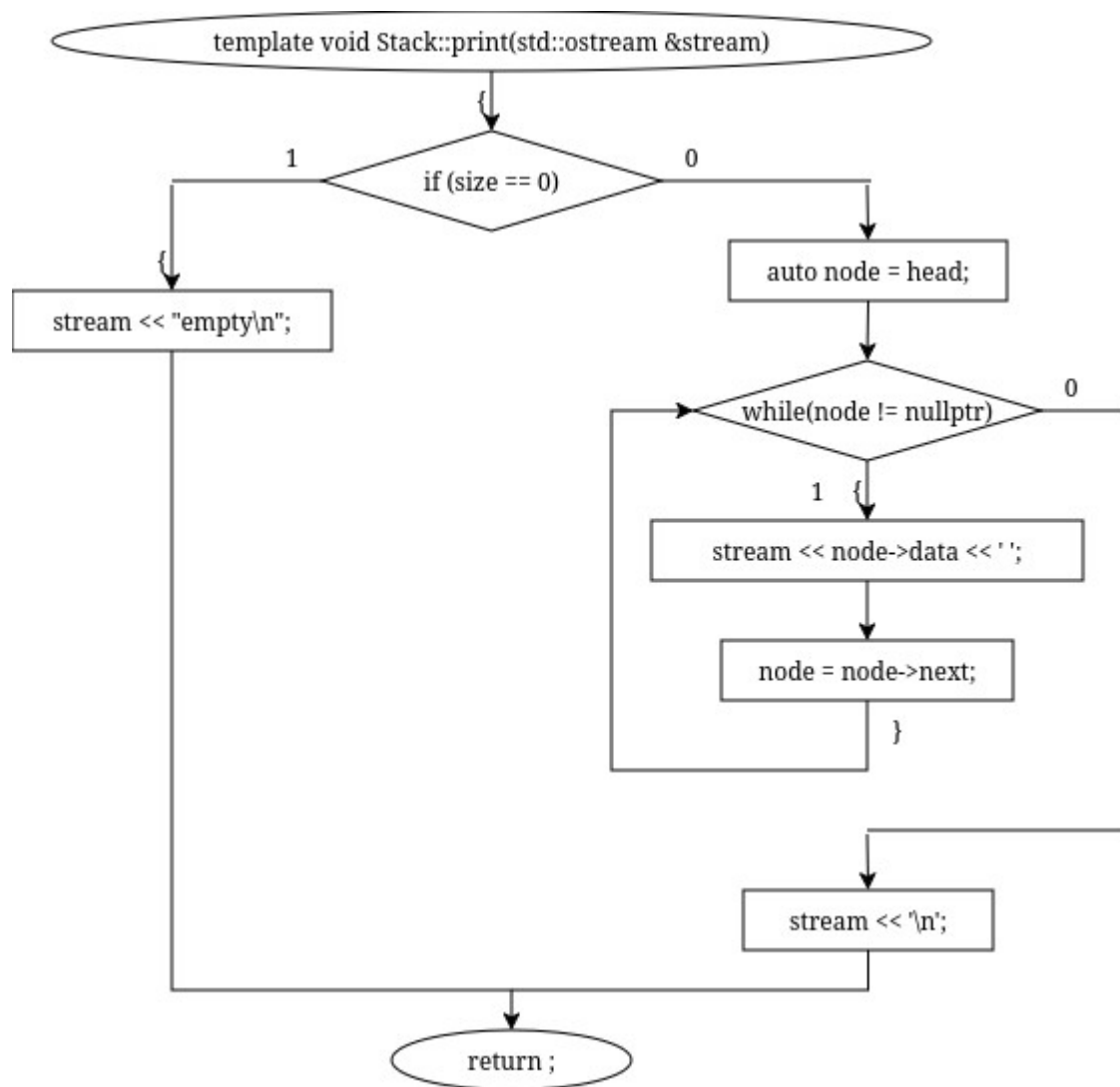


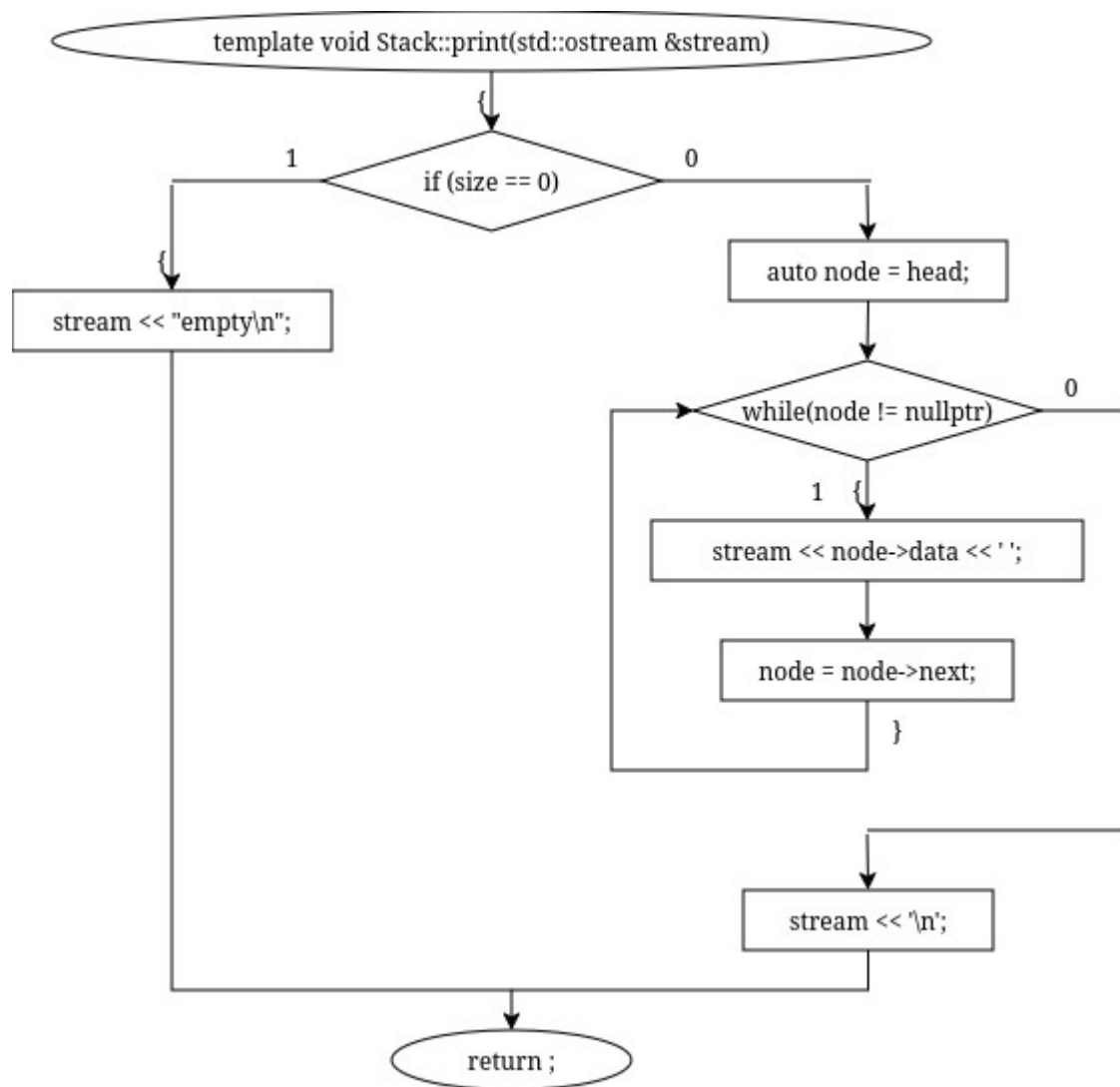












```
template<class T>
struct StackNode {
    T data;
    StackNode* next = nullptr;
};
```

```
template<class T>
struct StackResult {
    StackNode<T>* node = nullptr;
    int error = ok;
};
```

```
template<class T>
struct Stack {
    StackNode<T> * head = nullptr;
    int size = 0;

    void push(T&);
    void pop();
    StackResult<T> peek();

    void print(std::ostream&);
    void clear();
};
```

```
template<class T>
void Stack<T>::push(T &data) {
    auto node = new StackNode<T>;
    node->data = data;
    if (size == 0) {
        head = node;
        size++;
        return;
    }

    node->next = head;
    head = node;
    size++;
}
```

```
template<class T>
void Stack<T>::pop() {
    if (size == 0)
```

```

        return;

    auto del = head;
    head = head->next;

    delete del;
    size--;
}

template<class T>
StackResult<T> Stack<T>::peek() {
    StackResult<T> res;
    if (head == nullptr) {
        res.error = outOfRange;
        return res;
    }
    res.node = head;
    return res;
}

template<class T>
void Stack<T>::print(std::ostream &stream) {
    if (size == 0) {
        stream << "empty\n";
        return;
    }
    auto node = head;
    while (node != nullptr) {
        stream << node->data << ' ';
        node = node->next;
    }
    stream << '\n';
}

template<class T>
void Stack<T>::clear() {
    auto node = head;
    while (node != nullptr) {
        auto tmp = node->next;
        delete node;
        node = tmp;
    }
    head = nullptr;
    size = 0;
}

```

```

int mainStack() {
    using namespace std;
    Stack<string> stack;
    bool f = true;
    while (f) {
        cout << "a - add, c - clear, d - pop, e - exit, g - peek, p - print\n";
        char cmd;
        string s;
        cin >> cmd;
        switch (cmd) {
            case 'a':
                cout << "Enter value: ";
                cin >> s;
                stack.push(s);
                break;
            case 'c':
                stack.clear();
                break;
            case 'd':
                stack.pop();
                break;
            case 'e':
                f = false;
                break;
            case 'g': {
                auto peek = stack.peek();
                if (peek.error != 0) {
                    cout << "empty\n";
                    break;
                }
                cout << peek.node->data << '\n';
                break;
            }
            case 'p':
                stack.print(cout);
                break;
            default:
                cout << "command not found";
        }
    }
    stack.clear();
    return 0;
}

```

## РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРОГРАММЫ

Size:

6

1 2 3 4 5 6

1 2 3 4 5 6

Delete:

3

1 2 4 5 6

Key:

6

Index: 4

Amount:

2

First 2 words:

7 8

1 2 4 5 7 6

1 2 4 5 7 8 6

Second 2 words:

9 10

1 2 4 5 7 8 6 9

1 2 4 5 7 8 6 9 10

empty

1 2 4 5 7 8 6 9 10